МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра система автоматического проектирования

ОТЧЁТ

по лабораторной работе №1
по дисциплине «Алгоритмы и структуры данных»
Тема: «Алгоритмы сжатия без потерь»

Студент гр. 4354 _____ Чучалин И.В.
Преподаватель _____ Пестерев Д.О.

Санкт-Петербург
2026

Оглавление

Оглавление 2
Теоретическая часть 3
Практическая часть 6
Исходный код 13
Репозиторий на GitHub 13
Код программы 13

Теоретическая часть

Энтропия Шеннона

Энтропия Шеннона измеряет среднее количество информации на один символ источника.

Для дискретного источника с алфавитом A и вероятностями p_i энтропия вычисляется как:

$$H = -\sum_{i=1}^{|A|} p_i \log_2 p_i (\text{бит/символ})$$

При равномерном кодировании, когда каждый символ кодируется фиксированным числом байт M_s, верхняя граница энтропии составляет $8 \cdot M_s$ бит/символ. С увеличением M_s (1–4 байта) энтропия сообщения может меняться, так как меняется способ группировки байт в символы. Это важно при анализе эффективности последующего сжатия.

RLE

RLE заменяет повторяющиеся последовательности символов на пары: количество повторов и символ.

В расширенной версии если старший бит управляющего байта = 0 -идёт счётчик повторов (до 127), если старший бит = 1 -идёт длина неповторяющейся последовательности (до 127 символов).

Параметры M_s (длина символа в байтах) и M_c (длина управляющего символа) позволяют адаптировать алгоритм под разную структуру данных.

Временная сложность:

- Кодирование: $\Theta(n)$, один проход.
- Декодирование: $\Theta(m)$, где m -размер сжатых данных.

Пространственная сложность:

- Худший случай (нет повторов): $\Theta(2n)$ (каждый символ как литерал).
- Лучший случай (длинные серии): $\Theta(1)$ на выходе.

MTF

MTF преобразует входную последовательность в последовательность индексов в динамически изменяемом списке символов. Каждый использованный символ перемещается в начало списка.

Временная сложность:

- Кодирование: $\Theta(n)$, т.к. поиск в списке фиксированного размера (256) -константа.

- Декодирование: $\Theta(n)$.

Пространственная сложность:

- $\Theta(n)$ для выходных данных + $\Theta(1)$ для алфавита.

BWT

BWT переставляет символы блока так, чтобы одинаковые символы группировались. Прямое преобразование:

1. Построить все циклические сдвиги блока.
2. Отсортировать их лексикографически.
3. Взять последний столбец и запомнить индекс исходной строки.

Обратное преобразование восстанавливает блок по последнему столбцу и индексу.

Временная сложность (прямой метод с матрицей):

- Для блока размера m: $\Theta(m^2 \log\ m)$.
- Для файла из n байт, разбитого на блоки $k = n/m$: $\Theta(n \cdot m \log\ m)$.
- Обратное преобразование (сортировка подсчётом): $\Theta(n \log\ m)$.

Пространственная сложность:

- При хранении всей матрицы: $\Theta(m^2)$ на блок.
- При использовании суффиксного массива: $\Theta(m)$ на блок.

Для больших файлов BWT применяется блочно. Размер блока влияет на степень группировки

Прямое построение матрицы сдвигов неэффективно для больших блоков. Вместо этого строится суффиксный массив -массив начальных позиций всех суффиксов строки в лексикографическом порядке.

Последний столбец BWT получается как:

$\mathrm{BWT}[i] = \mathrm{data}[\mathrm{SA}[i] - 1]$ (с циклическим переходом)

Построение суффиксного массива может быть выполнено за $O(m)$ или $O(m \log\ m)$ в зависимости от реализации.

Алгоритм Хаффмана

Коды Хаффмана строятся на основе частот символов:

1. Подсчёт частот.
2. Построение бинарного дерева (объединение двух наименьших узлов).
3. Присвоение кодов (0 -левый, 1 -правый).

Временная сложность:

- Построение частот: $\Theta(n)$.
- Построение дерева: $\Theta(k\log\ k)$, где k -число уникальных символов.
- Итог: $\Theta(n + k\log\ k)$.

Пространственная сложность:

- Частоты и дерево: $\Theta(k)$.
- Сжатые данные: $\Theta(m)$.

Для декодирования необходимо хранить либо дерево, либо таблицу кодов. Для уменьшения метаданных используются канонические коды Хаффмана, которые строятся по длинам кодов и позволяют восстановить коды без хранения полного дерева.

Канонические коды Хаффмана обладают свойством: коды имеют одинаковую длину, образуют последовательные целые числа в двоичной системе.

Построение:

1. Определить длины кодов для каждого символа (из обычного дерева Хаффмана).
2. Отсортировать символы по длинам.
3. Присвоить коды возрастающими числами.

LZ77

LZ77 использует скользящее окно, разделённое на буфер поиска и буфер ввода. Каждый фрагмент кодируется тройкой: смещение назад в буфере поиска, длина совпадения, следующий символ (литерал).

Если совпадение не найдено -записывается литерал (смещение = 0, длина = 0).

Временная сложность:

- Кодирование: в наивной реализации $\Theta(n \cdot w \cdot l)$, где w -размер окна, l -буфер просмотра.
- Декодирование: $\Theta(n)$.

Пространственная сложность:

- $\Theta(w + l + n)$.

LZ78

LZ78 строит словарь фраз.

Каждый элемент кодируется парой: индекс предыдущей фразы в словаре, новый символ.

Временная сложность:

- Кодирование: в худшем случае $\Theta(n^2)$ (при поиске совпадений).
- Декодирование: $\Theta(n)$.

Пространственная сложность:

- Словарь может содержать до $O(n)$ фраз.

Для ограничения памяти вводят максимальный размер словаря.

Комбинированные компрессоры

В работе реализованы и исследованы следующие компрессоры:

1. HA – только Хаффман.
2. RLE – только RLE.
3. BWT + RLE.
4. BWT + MTF + HA.
5. BWT + MTF + RLE + HA.
6. LZ77 (или LZSS).
7. LZ77 + HA.
8. LZ78.
9. LZ78 + HA.

Практическая часть

Пример работы компрессоров:

```
========================================
Testing on: BW.raw
========================================

HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 89771 bytes
  Ratio: 18.70%
  Factor: 5.35:1
  Encode time: 203 ms
  Decode time: 83 ms
  Status: PASSED

RLE:
----------------------------------------
  Original: 480005 bytes
  Compressed: 20833 bytes
  Ratio: 4.34%
  Factor: 23.04:1
  Encode time: 55 ms
  Decode time: 334 ms
  Status: PASSED

BWT+RLE:
----------------------------------------
  Original: 480005 bytes
```

```
  Compressed: 35126 bytes
  Ratio: 7.32%
  Factor: 13.67:1
  Encode time: 12467 ms
  Decode time: 364 ms
  Status: PASSED

BWT+MTF+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 61499 bytes
  Ratio: 12.81%
  Factor: 7.81:1
  Encode time: 12558 ms
  Decode time: 178 ms
  Status: PASSED

BWT+MTF+RLE+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 17689 bytes
  Ratio: 3.69%
  Factor: 27.14:1
  Encode time: 12271 ms
  Decode time: 476 ms
  Status: PASSED

LZSS:
----------------------------------------
  Original: 480005 bytes
  Compressed: 124092 bytes
  Ratio: 25.85%
  Factor: 3.87:1
  Encode time: 173 ms
  Decode time: 53 ms
  Status: PASSED

LZSS+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 48440 bytes
  Ratio: 10.09%
  Factor: 9.91:1
  Encode time: 255 ms
  Decode time: 131 ms
  Status: PASSED

LZW:
----------------------------------------
  Original: 480005 bytes
  Compressed: 9911 bytes
  Ratio: 2.06%
  Factor: 48.43:1
  Encode time: 2012 ms
  Decode time: 32 ms
  Status: PASSED

LZW+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 10366 bytes
  Ratio: 2.16%
  Factor: 46.31:1
  Encode time: 2031 ms
  Decode time: 53 ms
  Status: PASSED

========================================
Testing on: Gray.raw
========================================

HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 136193 bytes
  Ratio: 28.37%
  Factor: 3.52:1
  Encode time: 284 ms
  Decode time: 131 ms
  Status: PASSED

RLE:
----------------------------------------
  Original: 480005 bytes
  Compressed: 161681 bytes
  Ratio: 33.68%
  Factor: 2.97:1
  Encode time: 71 ms
  Decode time: 372 ms
  Status: PASSED
```

```
BWT+RLE:
----------------------------------------
  Original: 480005 bytes
  Compressed: 195375 bytes
  Ratio: 40.70%
  Factor: 2.46:1
  Encode time: 13262 ms
  Decode time: 381 ms
  Status: PASSED

BWT+MTF+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 82861 bytes
  Ratio: 17.26%
  Factor: 5.79:1
  Encode time: 14194 ms
  Decode time: 208 ms
  Status: PASSED

BWT+MTF+RLE+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 98138 bytes
  Ratio: 20.45%
  Factor: 4.89:1
  Encode time: 14289 ms
  Decode time: 652 ms
  Status: PASSED

LZSS:
----------------------------------------
  Original: 480005 bytes
  Compressed: 176340 bytes
  Ratio: 36.74%
  Factor: 2.72:1
  Encode time: 1969 ms
  Decode time: 65 ms
  Status: PASSED

LZSS+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 105263 bytes
  Ratio: 21.93%
  Factor: 4.56:1
  Encode time: 2058 ms
  Decode time: 245 ms
  Status: PASSED

LZW:
----------------------------------------
  Original: 480005 bytes
  Compressed: 80914 bytes
  Ratio: 16.86%
  Factor: 5.93:1
  Encode time: 2171 ms
  Decode time: 160 ms
  Status: PASSED

LZW+HA:
----------------------------------------
  Original: 480005 bytes
  Compressed: 73147 bytes
  Ratio: 15.24%
  Factor: 6.56:1
  Encode time: 2192 ms
  Decode time: 299 ms
  Status: PASSED

========================================
Testing on: RGB.raw
========================================

HA:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 1016273 bytes
  Ratio: 70.57%
  Factor: 1.42:1
  Encode time: 1235 ms
  Decode time: 1911 ms
  Status: PASSED

RLE:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 1262086 bytes
  Ratio: 87.64%
  Factor: 1.14:1
  Encode time: 304 ms
```

```
  Decode time: 385 ms
  Status: PASSED

BWT+RLE:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 297628 bytes
  Ratio: 20.67%
  Factor: 4.84:1
  Encode time: 54320 ms
  Decode time: 1138 ms
  Status: PASSED

BWT+MTF+HA:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 247830 bytes
  Ratio: 17.21%
  Factor: 5.81:1
  Encode time: 54754 ms
  Decode time: 678 ms
  Status: PASSED

BWT+MTF+RLE+HA:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 182504 bytes
  Ratio: 12.67%
  Factor: 7.89:1
  Encode time: 53935 ms
  Decode time: 1693 ms
  Status: PASSED

LZSS:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 538485 bytes
  Ratio: 37.39%
  Factor: 2.67:1
  Encode time: 7087 ms
  Decode time: 196 ms
  Status: PASSED

LZSS+HA:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 368787 bytes
  Ratio: 25.61%
  Factor: 3.90:1
  Encode time: 7482 ms
  Decode time: 941 ms
  Status: PASSED

LZW:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 1084117 bytes
  Ratio: 75.29%
  Factor: 1.33:1
  Encode time: 7854 ms
  Decode time: 1902 ms
  Status: PASSED

LZW+HA:
----------------------------------------
  Original: 1440005 bytes
  Compressed: 838686 bytes
  Ratio: 58.24%
  Factor: 1.72:1
  Encode time: 9031 ms
  Decode time: 3619 ms
  Status: PASSED

========================================
Testing on: enwik7.txt
========================================

HA:
----------------------------------------
  Original: 10000000 bytes
  Compressed: 6412834 bytes
  Ratio: 64.13%
  Factor: 1.56:1
  Encode time: 8032 ms
  Decode time: 10305 ms
  Status: PASSED

RLE:
----------------------------------------
  Original: 10000000 bytes
  Compressed: 11436617 bytes
```

```
  Ratio: 114.37%
  Factor: 0.87:1
  Encode time: 2307 ms
  Decode time: 2902 ms
  Status: PASSED

BWT+RLE:
----------------------------------------
  Original: 10000000 bytes
  Compressed: 7934074 bytes
  Ratio: 79.34%
  Factor: 1.26:1
  Encode time: 847019 ms
  Decode time: 7874 ms
  Status: PASSED

BWT+MTF+HA:
----------------------------------------
  Original: 10000000 bytes
  Compressed: 3135738 bytes
  Ratio: 31.36%
  Factor: 3.19:1
  Encode time: 878909 ms
  Decode time: 7827 ms
  Status: PASSED
```

Задания

Задание 1. RLE

1. Обработка неповторяющихся последовательностей

- Изначально: Управляющий байт (Mc=1) мог хранить только длину последовательности (0-255). Для литералов использовался отдельный режим без оптимизации.
- Изменение: В коде (rle.cpp) для литералов используется управляющий байт со старшим битом, равным 1. Это позволяет кодировать неповторяющиеся последовательности. Максимальная длина последовательности, которую можно записать -2^(Mc*8) - 1.

2. Выбор Ms и Mc для изображений (raw формат)

- Черно-белое (1 бит/пиксель): Ms = 1 (байт), Mc = 2 (чтобы эффективно кодировать длинные серии одинаковых байт 0x00/0xFF).
- Grayscale (1 байт/пиксель): Ms = 1 (байт), Mc = 2 (аналогично Ч/Б).
- Цветное (3 байта/пиксель): Ms = 3 (байта). Mc рекомендуется 1 или 2.

  Выбор Ms=3 позволяет кодировать повторения целых пикселей, что эффективно для областей с однородным цветом.

3. Проблема с UTF-8 и её решение

- Проблема: В UTF-8 коды символов имеют переменную длину. RLE оперирует байтами. Кодирование на уровне байтов может сломать мультибайтные символы, т.к. последовательность байтов, составляющих один символ, будет обрабатываться как независимые литералы.
- Решение: Перед сжатием преобразовать строку в последовательность кодов символов фиксированного размера , и только затем применять RLE, установив Ms = 4.

4. Метаданные в закодированном файле

Файл, создаваемый RLE, содержит заголовок:

- Ms (1 байт) -длина кода символа.
- Mc (1 байт) -длина управляющего кода.
- originalSize (8 байт) -исходный размер данных.
- encodedSize (8 байт) -размер закодированных данных.

  Это гарантирует корректное декодирование.

  5. Оценка коэффициента сжатия
- Оценка в коде (analyzeFile) основана на анализе повторяющихся последовательностей.
- Фактический коэффициент вычисляется после кодирования: (encodedSize + headerSize) / originalSize.
- Для энтропийного анализа используется формула Шеннона:

  $H = -\Sigma(p * \log_2(p))$ (entropy.cpp).

Задание 2. Энтропийное кодирование и BWT

1. Энтропия от длины кода символа

С увеличением Ms энтропия на символ растет, а количество уникальных символов увеличивается. Энтропия, выраженная в битах на байт, будет уменьшаться, если в данных есть структура на уровне групп байтов. Результат показывает, что данные не являются полностью случайными на более высоких уровнях агрегации.

2. Влияние MTF на энтропию

Алгоритм MTF (mtf.cpp) заменяет повторяющиеся символы на малые числа, группируя вероятностное распределение. Это увеличивает энтропию, но улучшает сжатие последующим энтропийным кодером, так как создает последовательности с преобладанием малых чисел, которые получают короткие коды.

3. Метаданные для кодов Хаффмана

- Классический Хаффман (huffman.cpp): Необходимо сохранить структуру дерева. В коде используется обход в глубину: 0 -узел, 1 -лист, за которым следует байт символа.
- Канонический Хаффман (huffman_canonical.cpp): Для декодирования достаточно сохранить длины кодов для каждого символа (256 байт). Это значительно уменьшает размер метаданных по сравнению с полным деревом.

4. Проблема арифметического кодирования с double

При использовании типа double для хранения границ интервала (левая low, правая high) точности хватает примерно на 50-60 бит. При длине сообщения более 30-40 символов разница между low и high становится исчезающе малой, и границы могут совпасть при сравнении. Это делает double непригодным для длинных сообщений; требуются

целочисленные реализации с расширенной точностью.

5. Сложность BWT с прямым построением матрицы

- Временная сложность: $O(n^2 \log n)$ из-за построения матрицы n x n и её сортировки. Для строки "banana" (n=6) это допустимо, но для файлов в мегабайты -нет.
- Пространственная сложность: $O(n^2)$.

6. Сложность BWT с использованием суффиксного массива (buildSuffixArrayDoubling)

- Временная сложность: $O(n \log n)$.
- Пространственная сложность: $O(n)$. Реализованный в bwt.cpp алгоритм удвоения (buildSuffixArrayDoubling) использует несколько массивов размера 2n для временных вычислений, что дает $O(n)$ памяти.

7. Эффективность BWT+RLE

Последовательное применение BWT и RLE (функция testBWTAndRLE) эффективно для текстов, так как BWT группирует одинаковые символы, создавая длинные серии. Это значительно улучшает коэффициент сжатия по сравнению с применением одного RLE. Для русского текста и enwik7 ожидается значительное снижение размера.

Задание 3. LZ-методы

1. Метаданные для LZ77/LZSS

Сжатое сообщение сохраняется в виде последовательности токенов. Для корректного декодирования необходимы:

- Размер исходных данных (originalSize).
- Для LZ77/LZSS: размер окна (windowSize) и буфера просмотра (lookaheadSize).
- Контрольная сумма (checksum).
- Структура заголовка (LZ77Header / LZSSHeader).

2. Адаптация LZ78 с ограничением словаря

В функции encodeTokens (файл lz78.cpp) максимальный размер словаря (maxDictSize) ограничивает количество добавляемых фраз. По достижении лимита новые фразы перестают добавляться, что позволяет контролировать использование памяти.

3. Начальный словарь для LZW

Для декодирования LZW не нужно сохранять начальный словарь отдельно, так как он всегда одинаков и инициализируется всеми возможными байтами (0-255). Это реализовано в функциях buildInitialDictionary и decodeTokens. Словарь восстанавливается в процессе декодирования.

4. Влияние размера буфера/словаря

- LZSS: Увеличение windowSize позволяет находить более длинные и отдаленные совпадения, улучшая сжатие, но увеличивая время поиска.

Большой lookaheadSize тоже улучшает сжатие, но замедляет работу.

- LZW: Увеличение maxDictSize (до 4096, 65536) позволяет кодировать более длинные фразы, улучшая сжатие на больших файлах. При слишком малом словаре словарь быстро заполняется, и алгоритм перестает добавлять новые фразы, что снижает коэффициент сжатия.

Задание 4. Исследования и итоговые компрессоры

1. Оптимальный размер блока для BWT+MTF

Оптимальный размер блока выбирается на основе энтропийного анализа (entropy.cpp). Для текстов (enwik7, русский) с увеличением блока энтропия после BWT+MTF снижается, так как алгоритм лучше выявляет повторяющиеся паттерны. На графике будет видно, что при превышении определенного порога улучшение сжатия замедляется, а время работы резко растет. Оптимальный блок -компромисс между коэффициентом сжатия и скоростью.

2. Зависимость коэффициента сжатия LZSS от размера буфера

Коэффициент сжатия растет с увеличением размера буфера. График покажет логарифмическую зависимость: быстрый рост при малых размерах буфера (до 1024 байт) и последующее замедление. Размер буфера в 4096 байт часто является хорошим компромиссом, обеспечивая высокое сжатие без значительного снижения скорости.

3. Зависимость коэффициента сжатия LZW от размера словаря

График зависимости коэффициента сжатия от размера словаря (от 256 до 65536) показывает быстрый рост до 4096 (12-битные коды) и дальнейшее замедление. После 8192-16384 прирост коэффициента незначителен, а требования к памяти растут.

Исходный код

Репозиторий на GitHub: https://github.com/tethranialn/AISD_2.2_n1
Код программы (без заголовочных файлов):

analysis.cpp

```
#include "analysis.h"
#include "raw_convert.h"
#include "rle.h"
#include "entropy.h"
#include "mtf.h"
#include "huffman.h"
#include "huffman_canonical.h"
#include "bwt.h"
#include "lz77.h"
#include "lzss.h"
#include "lz78.h"
#include "lzw.h"
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cstring>
#include <sys/stat.h>
#include <chrono>
#include <functional>
#include <map>

using namespace std;
```

```
using namespace chrono;

bool Analysis::fileExists(const string& path) {
    struct stat buffer;
    return (stat(path.c_str(), &buffer) == 0);
}

void Analysis::printFileSelectionMenu() {
    cout << "\n----------------------------------------\n";
    cout << "Select files to process:\n";
    cout << "1. BW.raw (Black & White image)\n";
    cout << "2. Gray.raw (Grayscale image)\n";
    cout << "3. RGB.raw (Color RGB image)\n";
    cout << "4. enwik7.txt (English text)\n";
    cout << "5. RusText.txt (Russian text)\n";
    cout << "6. BIN.bin (Binary data)\n";
    cout << "7. All files\n";
    cout << "8. Cancel\n";
    cout << "Enter your choice (1-8): ";
}

vector<string> Analysis::getSelectedFiles(const string& inputDir, int choice) {
    vector<string> allFiles = { "BW.raw", "Gray.raw", "RGB.raw", "enwik7.txt", "RusText.txt", "BIN.bin" };
    vector<string> selectedFiles;

    if (choice == 7) {
        return allFiles;
    }

    if (choice >= 1 && choice <= 6) {
        selectedFiles.push_back(allFiles[choice - 1]);
    }

    return selectedFiles;
}

void Analysis::rleAnalysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "RLE Compression Analysis\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int choice;
    cin >> choice;

    if (choice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, choice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    struct TestConfig {
        string filename;
        uint8_t Ms;
        uint8_t Mc;
        string description;
        bool isRaw;
    };

    vector<TestConfig> allTests = {
        {"BW.raw", 1, 1, "Black & White", true},
        {"BW.raw", 1, 2, "Black & White", true},
        {"Gray.raw", 1, 1, "Grayscale", true},
        {"Gray.raw", 1, 2, "Grayscale", true},
        {"RGB.raw", 3, 1, "Color RGB", true},
        {"RGB.raw", 3, 2, "Color RGB", true},
        {"enwik7.txt", 1, 1, "Text (English)", false},
        {"RusText.txt", 1, 1, "Text (Russian)", false},
        {"BIN.bin", 1, 1, "Binary", false}
    };

    for (const auto& filename : files) {
        for (const auto& test : allTests) {
            if (test.filename != filename) continue;

            string inputPath = inputDir + "\\" + test.filename;

            if (!fileExists(inputPath)) {
                cout << "File not found: " << test.filename << " - skipping\n\n";
                continue;
            }

            cout << "File: " << test.filename << " (" << test.description << ")\n";
            cout << "Parameters: Ms=" << (int)test.Ms << ", Mc=" << (int)test.Mc << "\n";
            cout << "----------------------------------------\n";
```

```
            string encodedPath = encodedDir + "\\" + test.filename + ".rle";
            string decodedPath = decodedDir + "\\" + test.filename;

            string errorMsg;

            if (test.isRaw) {
                uint8_t type = 0;
                uint32_t pixels = 0;
                vector<uint8_t> pixelData;

                ifstream rawFile(inputPath, ios::binary);
                if (rawFile.is_open()) {
                    rawFile.read((char*)&type, 1);
                    rawFile.read((char*)&pixels, 4);
                    pixelData.assign((istreambuf_iterator<char>(rawFile)), istreambuf_iterator<char>());
                    rawFile.close();
                }

                vector<uint8_t> encodedPixelData = RLE::encode(pixelData, test.Ms, test.Mc);

                RLEHeader header;
                header.Ms = test.Ms;
                header.Mc = test.Mc;
                header.originalSize = pixelData.size();
                header.encodedSize = encodedPixelData.size();

                size_t encodedTotalSize = sizeof(header) + encodedPixelData.size();

                ofstream outFile(encodedPath, ios::binary);
                if (outFile.is_open()) {
                    outFile.write(reinterpret_cast<const char*>(&header.Ms), sizeof(header.Ms));
                    outFile.write(reinterpret_cast<const char*>(&header.Mc), sizeof(header.Mc));
                    outFile.write(reinterpret_cast<const char*>(&header.originalSize), sizeof(header.originalSize));
                    outFile.write(reinterpret_cast<const char*>(&header.encodedSize), sizeof(header.encodedSize));
                    outFile.write(reinterpret_cast<const char*>(encodedPixelData.data()), encodedPixelData.size());
                    outFile.close();
                }

                vector<uint8_t> decodedPixelData = RLE::decode(encodedPixelData, test.Ms, test.Mc);

                ofstream decFile(decodedPath, ios::binary);
                if (decFile.is_open()) {
                    decFile.put(type);
                    decFile.write((char*)&pixels, 4);
                    decFile.write((char*)decodedPixelData.data(), decodedPixelData.size());
                    decFile.close();
                }

                if (pixelData.size() == decodedPixelData.size() &&
                    memcmp(pixelData.data(), decodedPixelData.data(), pixelData.size()) == 0) {
                    double ratio = (double)encodedTotalSize / pixelData.size() * 100.0;
                    double factor = (double)pixelData.size() / encodedTotalSize;
                    cout << "  Pixel data: " << pixelData.size() << " bytes\n";
                    cout << "  Encoded:    " << encodedTotalSize << " bytes\n";
                    cout << "  Ratio:      " << fixed << setprecision(2) << ratio << "% of pixel data\n";
                    cout << "  Factor:     " << fixed << setprecision(2) << factor << ":1\n";
                    cout << "  Status:     PASSED\n\n";
                }
                else {
                    cout << "  Status:     FAILED\n\n";
                }
            }
            else {
                CompressionStats actual = RLE::compressFile(inputPath, encodedPath, test.Ms, test.Mc, errorMsg);
                if (!errorMsg.empty()) {
                    cout << "Error: " << errorMsg << "\n\n";
                    continue;
                }

                bool verified = RLE::verifyCycle(inputPath, encodedPath, decodedPath, test.Ms, test.Mc, errorMsg);

                cout << "  Original: " << actual.originalSize << " bytes\n";
                cout << "  Encoded:  " << actual.encodedSize << " bytes\n";
                cout << "  Ratio:    " << fixed << setprecision(2) << actual.compressionRatio << "% of original\n";
                cout << "  Factor:   " << fixed << setprecision(2) << actual.compressionFactor << ":1\n";
                cout << "  Status:   " << (verified ? "PASSED" : "FAILED") << "\n\n";
            }
        }
    }
}

void Analysis::entropyAnalysis(const string& inputDir) {
    cout << "\n========================================\n";
    cout << "Entropy Calculation\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int choice;
    cin >> choice;
```

```
    if (choice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, choice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string filePath = inputDir + "\\" + filename;

        if (!fileExists(filePath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "File: " << filename << "\n";
        cout << "----------------------------------------\n";

        ifstream file(filePath, ios::binary);
        if (!file.is_open()) {
            cout << "Error: Cannot open file\n\n";
            continue;
        }

        vector<uint8_t> data((istreambuf_iterator<char>(file)), istreambuf_iterator<char>());
        file.close();

        if (data.empty()) {
            cout << "File is empty\n\n";
            continue;
        }

        cout << "File size: " << data.size() << " bytes\n\n";
        cout << "Entropy depending on symbol code length (Ms):\n";
        cout << "Ms (bytes) | Entropy H(X) (bits/symbol)\n";
        cout << "----------------------------------------\n";

        for (uint8_t Ms = 1; Ms <= 4; Ms++) {
            EntropyResult result = EntropyCalculator::calculate(data, Ms);

            cout << "   " << setw(2) << (int)Ms << "        |   "
                << fixed << setprecision(4) << result.entropy << "\n";
        }

        cout << "\n";
    }
}

void Analysis::mtfAnalysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "MTF Transform Analysis\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int choice;
    cin >> choice;

    if (choice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, choice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;

        if (!fileExists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "File: " << filename << "\n";
        cout << "----------------------------------------\n";

        ifstream inFile(inputPath, ios::binary);
        if (!inFile.is_open()) {
            cout << "Error: Cannot open file\n\n";
            continue;
        }
```

```
        vector<uint8_t> originalData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();

        cout << "Original size: " << originalData.size() << " bytes\n";

        vector<uint8_t> encodedData = MTF::encode(originalData);

        string encodedPath = encodedDir + "\\" + filename + ".mtf";
        ofstream encFile(encodedPath, ios::binary);
        encFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
        encFile.close();

        cout << "MTF encoded size: " << encodedData.size() << " bytes\n";

        vector<uint8_t> decodedData = MTF::decode(encodedData);

        string decodedPath = decodedDir + "\\" + filename;
        ofstream decFile(decodedPath, ios::binary);
        decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
        decFile.close();

        if (originalData.size() == decodedData.size() &&
            memcmp(originalData.data(), decodedData.data(), originalData.size()) == 0) {
            cout << "Verification: PASSED\n";
        }
        else {
            cout << "Verification: FAILED\n";
        }

        cout << "\n";
    }
}

void Analysis::huffmanAnalysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "Huffman Compression Analysis\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int choice;
    cin >> choice;

    if (choice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, choice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;

        if (!fileExists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        string errorMsg;
        CompressionStatsHuffman stats = Huffman::analyzeFile(inputPath, errorMsg);

        if (!errorMsg.empty()) {
            cout << "Error analyzing " << filename << ": " << errorMsg << "\n\n";
            continue;
        }

        Huffman::printStats(stats);

        string encodedPath = encodedDir + "\\" + filename + ".huf";
        string decodedPath = decodedDir + "\\" + filename;

        if (Huffman::encodeFileWithStats(inputPath, encodedPath, stats, errorMsg)) {
            cout << "Encoding successful\n";

            if (Huffman::decodeFileWithVerify(encodedPath, decodedPath, errorMsg)) {
                cout << "Decoding and verification: PASSED\n";
            }
            else {
                cout << "Decoding verification: FAILED - " << errorMsg << "\n";
            }
        }
        else {
            cout << "Encoding failed: " << errorMsg << "\n";
        }

        cout << "\n";
    }
```

```
}

void Analysis::huffmanCanonicalAnalysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "Canonical Huffman Compression\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int fileChoice;
    cin >> fileChoice;

    if (fileChoice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, fileChoice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;

        if (!fileExists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "\nFile: " << filename << "\n";
        cout << "----------------------------------------\n";

        string errorMsg;

        string encodedPath = encodedDir + "\\" + filename + ".huf_canonical";
        string decodedPath = decodedDir + "\\" + filename;

        ifstream inFile(inputPath, ios::binary);
        vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();

        cout << "Original size: " << inputData.size() << " bytes\n";

        auto start = high_resolution_clock::now();
        bool success = HuffmanCanonical::encodeFile(inputPath, encodedPath, errorMsg);
        auto end = high_resolution_clock::now();
        auto encodeTime = duration_cast<milliseconds>(end - start).count();

        if (!success) {
            cout << "Encoding failed: " << errorMsg << "\n\n";
            continue;
        }

        ifstream encodedFile(encodedPath, ios::binary);
        vector<uint8_t> encodedData((istreambuf_iterator<char>(encodedFile)), istreambuf_iterator<char>());
        encodedFile.close();

        cout << "Compressed size: " << encodedData.size() << " bytes\n";
        cout << "Compression ratio: " << fixed << setprecision(2)
            << (double)encodedData.size() / inputData.size() * 100.0 << "%\n";
        cout << "Encoding time: " << encodeTime << " ms\n";

        start = high_resolution_clock::now();
        bool verified = HuffmanCanonical::verifyCycle(inputPath, encodedPath, decodedPath, errorMsg);
        end = high_resolution_clock::now();
        auto decodeTime = duration_cast<milliseconds>(end - start).count();

        cout << "Decoding time: " << decodeTime << " ms\n";
        cout << "Status: " << (verified ? "PASSED" : "FAILED") << "\n\n";
    }
}

void Analysis::suffixArrayToBWTDemo(const string& inputDir) {
    cout << "\n========================================\n";
    cout << "Suffix Array to BWT Last Column Demo\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int choice;
    cin >> choice;

    if (choice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, choice);

    if (files.empty()) {
```

```
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string filePath = inputDir + "\\" + filename;

        if (!fileExists(filePath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "\nFile: " << filename << "\n";
        cout << "----------------------------------------\n";

        ifstream inFile(filePath, ios::binary);
        if (!inFile.is_open()) {
            cout << "Error: Cannot open file\n\n";
            continue;
        }

        vector<uint8_t> fileData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();

        cout << "Size: " << fileData.size() << " bytes\n";

        size_t demoSize = min(fileData.size(), size_t(50));
        vector<uint8_t> demoData(fileData.begin(), fileData.begin() + demoSize);

        if (demoSize < fileData.size()) {
            cout << "Using first " << demoSize << " bytes\n";
        }

        cout << "\nInput data (first " << demoSize << " bytes):\n";
        for (size_t i = 0; i < demoSize; i++) {
            if (isprint(demoData[i])) {
                cout << (char)demoData[i];
            }
            else {
                cout << ".";
            }
        }
        cout << "\n\n";

        vector<int32_t> suffixArray = BWT::buildSuffixArrayDoubling(demoData);

        cout << "Suffix array (indices of cyclic rotations in sorted order):\n";
        for (size_t i = 0; i < suffixArray.size(); i++) {
            cout << "  [" << setw(2) << i << "] = " << suffixArray[i];
            cout << "  -> rotation: ";
            for (size_t j = 0; j < min(demoSize, size_t(20)); j++) {
                size_t pos = (suffixArray[i] + j) % demoSize;
                if (isprint(demoData[pos])) {
                    cout << (char)demoData[pos];
                }
                else {
                    cout << ".";
                }
            }
            if (demoSize > 20) cout << "...";
            cout << "\n";
        }

        vector<uint8_t> lastColumn = BWT::suffixArrayToLastColumn(demoData, suffixArray);

        cout << "\nBWT last column:\n";
        for (size_t i = 0; i < lastColumn.size(); i++) {
            if (isprint(lastColumn[i])) {
                cout << (char)lastColumn[i];
            }
            else {
                cout << "0x" << hex << setw(2) << setfill('0') << (int)lastColumn[i] << dec;
            }
        }
        cout << "\n";

        cout << "\nLast column bytes:\n";
        for (size_t i = 0; i < lastColumn.size(); i++) {
            cout << "0x" << hex << setw(2) << setfill('0') << (int)lastColumn[i] << " ";
            if ((i + 1) % 16 == 0) cout << "\n";
        }
        cout << dec << "\n\n";
    }
}

void Analysis::testBWTAndRLE(const string& inputDir) {
    printFileSelectionMenu();
    int fileChoice;
    cin >> fileChoice;
    if (fileChoice == 8) {
        cout << "Operation cancelled\n";
```

```
        return;
    }
    vector<string> files = getSelectedFiles(inputDir, fileChoice);
    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }
    BWT::testBWTAndRLE(inputDir, files);
}

void Analysis::testEfficientBWT(const string& inputDir) {
    BWT::testEfficientBWT(inputDir, getSelectedFiles(inputDir, 7));
}

void Analysis::testOnBanana() {
    BWT::testOnBanana();
}

void Analysis::lz77Analysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "LZ77 Compression Analysis\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int fileChoice;
    cin >> fileChoice;

    if (fileChoice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, fileChoice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;

        if (!fileExists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "\nFile: " << filename << "\n";
        cout << "----------------------------------------\n";

        string errorMsg;

        string encodedPath = encodedDir + "\\" + filename + ".lz77";
        string decodedPath = decodedDir + "\\" + filename;

        LZ77Stats stats = LZ77::compressFile(inputPath, encodedPath, 4096, 16, errorMsg);

        if (!errorMsg.empty()) {
            cout << "Error: " << errorMsg << "\n\n";
            continue;
        }

        LZ77::printStats(stats);

        bool verified = LZ77::verifyCycle(inputPath, encodedPath, decodedPath, 4096, 16, errorMsg);

        cout << "  Status: " << (verified ? "PASSED" : "FAILED") << "\n\n";
    }
}

void Analysis::lzssAnalysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "LZSS Compression Analysis\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int fileChoice;
    cin >> fileChoice;

    if (fileChoice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, fileChoice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }
```

```
    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;

        if (!fileExists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "\nFile: " << filename << "\n";
        cout << "----------------------------------------\n";

        string errorMsg;

        string encodedPath = encodedDir + "\\" + filename + ".lzss";
        string decodedPath = decodedDir + "\\" + filename;

        LZSSStats stats = LZSS::compressFile(inputPath, encodedPath, 4096, 16, errorMsg);

        if (!errorMsg.empty()) {
            cout << "Error: " << errorMsg << "\n\n";
            continue;
        }

        LZSS::printStats(stats);

        bool verified = LZSS::verifyCycle(inputPath, encodedPath, decodedPath, 4096, 16, errorMsg);

        cout << "  Status: " << (verified ? "PASSED" : "FAILED") << "\n\n";
    }
}

void Analysis::lz78Analysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "LZ78 Compression Analysis\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int fileChoice;
    cin >> fileChoice;

    if (fileChoice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, fileChoice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;

        if (!fileExists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "\nFile: " << filename << "\n";
        cout << "----------------------------------------\n";

        string errorMsg;

        string encodedPath = encodedDir + "\\" + filename + ".lz78";
        string decodedPath = decodedDir + "\\" + filename;

        LZ78Stats stats = LZ78::compressFile(inputPath, encodedPath, 4096, errorMsg);

        if (!errorMsg.empty()) {
            cout << "Error: " << errorMsg << "\n\n";
            continue;
        }

        LZ78::printStats(stats);

        bool verified = LZ78::verifyCycle(inputPath, encodedPath, decodedPath, 4096, errorMsg);

        cout << "  Status: " << (verified ? "PASSED" : "FAILED") << "\n\n";
    }
}

void Analysis::lzwAnalysis(const string& inputDir, const string& encodedDir, const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "LZW Compression Analysis\n";
    cout << "========================================\n\n";

    printFileSelectionMenu();
    int fileChoice;
    cin >> fileChoice;
```

```
    if (fileChoice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> files = getSelectedFiles(inputDir, fileChoice);

    if (files.empty()) {
        cout << "Invalid choice\n";
        return;
    }

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;

        if (!fileExists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "\nFile: " << filename << "\n";
        cout << "----------------------------------------\n";

        string errorMsg;

        string encodedPath = encodedDir + "\\" + filename + ".lzw";
        string decodedPath = decodedDir + "\\" + filename;

        LZWStats stats = LZW::compressFile(inputPath, encodedPath, 4096, errorMsg);

        if (!errorMsg.empty()) {
            cout << "Error: " << errorMsg << "\n\n";
            continue;
        }

        LZW::printStats(stats);

        bool verified = LZW::verifyCycle(inputPath, encodedPath, decodedPath, 4096, errorMsg);

        cout << "  Status: " << (verified ? "PASSED" : "FAILED") << "\n\n";
    }
}

void Analysis::runSingleAnalysis(const string& algorithm,
    const string& inputDir,
    const string& encodedDir,
    const string& decodedDir,
    vector<AlgorithmResult>& results) {
    vector<string> files = { "BW.raw", "Gray.raw", "RGB.raw", "enwik7.txt", "RusText.txt", "BIN.bin" };

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;
        if (!fileExists(inputPath)) {
            AlgorithmResult res;
            res.algorithm = algorithm;
            res.filename = filename;
            res.success = false;
            res.details = "File not found";
            results.push_back(res);
            continue;
        }

        AlgorithmResult res;
        res.algorithm = algorithm;
        res.filename = filename;

        auto start = high_resolution_clock::now();

        try {
            if (algorithm == "RLE") {
                string encodedPath = encodedDir + "\\" + filename + ".rle";
                string decodedPath = decodedDir + "\\" + filename;
                string errorMsg;

                CompressionStats stats = RLE::compressFile(inputPath, encodedPath, 1, 2, errorMsg);
                if (errorMsg.empty()) {
                    bool verified = RLE::verifyCycle(inputPath, encodedPath, decodedPath, 1, 2, errorMsg);
                    res.success = verified;
                    res.ratio = stats.compressionRatio;
                    res.originalSize = stats.originalSize;
                    res.processedSize = stats.encodedSize;
                    res.details = verified ? "PASSED" : "FAILED";
                }
                else {
                    res.success = false;
                    res.details = errorMsg;
                }
            }
            else if (algorithm == "MTF") {
                ifstream inFile(inputPath, ios::binary);
```

```
        vector<uint8_t> originalData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();

        vector<uint8_t> encodedData = MTF::encode(originalData);

        string encodedPath = encodedDir + "\\" + filename + ".mtf";
        ofstream encFile(encodedPath, ios::binary);
        encFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
        encFile.close();

        vector<uint8_t> decodedData = MTF::decode(encodedData);

        string decodedPath = decodedDir + "\\" + filename;
        ofstream decFile(decodedPath, ios::binary);
        decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
        decFile.close();

        res.success = (originalData.size() == decodedData.size() &&
            memcmp(originalData.data(), decodedData.data(), originalData.size()) == 0);
        res.originalSize = originalData.size();
        res.processedSize = encodedData.size();
        res.ratio = (double)encodedData.size() / originalData.size() * 100.0;
        res.details = res.success ? "PASSED" : "FAILED";
    }
    else if (algorithm == "Huffman") {
        string errorMsg;
        CompressionStatsHuffman stats = Huffman::analyzeFile(inputPath, errorMsg);
        if (errorMsg.empty()) {
            string encodedPath = encodedDir + "\\" + filename + ".huf";
            string decodedPath = decodedDir + "\\" + filename;

            if (Huffman::encodeFileWithStats(inputPath, encodedPath, stats, errorMsg)) {
                bool verified = Huffman::decodeFileWithVerify(encodedPath, decodedPath, errorMsg);
                res.success = verified;
                res.ratio = stats.compressionRatio;
                res.originalSize = stats.originalSize;
                res.processedSize = stats.encodedSize;
                res.details = verified ? "PASSED" : "FAILED";
            }
            else {
                res.success = false;
                res.details = errorMsg;
            }
        }
        else {
            res.success = false;
            res.details = errorMsg;
        }
    }
    else if (algorithm == "Canonical Huffman") {
        string encodedPath = encodedDir + "\\" + filename + ".huf_canonical";
        string decodedPath = decodedDir + "\\" + filename;
        string errorMsg;

        ifstream inFile(inputPath, ios::binary);
        vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();
        res.originalSize = inputData.size();

        bool success = HuffmanCanonical::encodeFile(inputPath, encodedPath, errorMsg);

        if (success) {
            ifstream encodedFile(encodedPath, ios::binary);
            vector<uint8_t> encodedData((istreambuf_iterator<char>(encodedFile)), istreambuf_iterator<char>());
            encodedFile.close();
            res.processedSize = encodedData.size();
            res.ratio = (double)encodedData.size() / res.originalSize * 100.0;

            bool verified = HuffmanCanonical::verifyCycle(inputPath, encodedPath, decodedPath, errorMsg);
            res.success = verified;
            res.details = verified ? "PASSED" : "FAILED";
        }
        else {
            res.success = false;
            res.details = errorMsg;
        }
    }
    else if (algorithm == "LZ77") {
        string encodedPath = encodedDir + "\\" + filename + ".lz77";
        string decodedPath = decodedDir + "\\" + filename;
        string errorMsg;

        LZ77Stats stats = LZ77::compressFile(inputPath, encodedPath, 4096, 16, errorMsg);
        if (errorMsg.empty()) {
            bool verified = LZ77::verifyCycle(inputPath, encodedPath, decodedPath, 4096, 16, errorMsg);
            res.success = verified;
            res.ratio = stats.compressionRatio;
            res.originalSize = stats.originalSize;
            res.processedSize = stats.compressedSize;
            res.details = verified ? "PASSED" : "FAILED";
        }
```

```
        else {
            res.success = false;
            res.details = errorMsg;
        }
    }
    else if (algorithm == "LZSS") {
        string encodedPath = encodedDir + "\\" + filename + ".lzss";
        string decodedPath = decodedDir + "\\" + filename;
        string errorMsg;

        LZSSStats stats = LZSS::compressFile(inputPath, encodedPath, 4096, 16, errorMsg);
        if (errorMsg.empty()) {
            bool verified = LZSS::verifyCycle(inputPath, encodedPath, decodedPath, 4096, 16, errorMsg);
            res.success = verified;
            res.ratio = stats.compressionRatio;
            res.originalSize = stats.originalSize;
            res.processedSize = stats.compressedSize;
            res.details = verified ? "PASSED" : "FAILED";
        }
        else {
            res.success = false;
            res.details = errorMsg;
        }
    }
    else if (algorithm == "LZ78") {
        string encodedPath = encodedDir + "\\" + filename + ".lz78";
        string decodedPath = decodedDir + "\\" + filename;
        string errorMsg;

        LZ78Stats stats = LZ78::compressFile(inputPath, encodedPath, 4096, errorMsg);
        if (errorMsg.empty()) {
            bool verified = LZ78::verifyCycle(inputPath, encodedPath, decodedPath, 4096, errorMsg);
            res.success = verified;
            res.ratio = stats.compressionRatio;
            res.originalSize = stats.originalSize;
            res.processedSize = stats.compressedSize;
            res.details = verified ? "PASSED" : "FAILED";
        }
        else {
            res.success = false;
            res.details = errorMsg;
        }
    }
    else if (algorithm == "LZW") {
        string encodedPath = encodedDir + "\\" + filename + ".lzw";
        string decodedPath = decodedDir + "\\" + filename;
        string errorMsg;

        LZWStats stats = LZW::compressFile(inputPath, encodedPath, 4096, errorMsg);
        if (errorMsg.empty()) {
            bool verified = LZW::verifyCycle(inputPath, encodedPath, decodedPath, 4096, errorMsg);
            res.success = verified;
            res.ratio = stats.compressionRatio;
            res.originalSize = stats.originalSize;
            res.processedSize = stats.compressedSize;
            res.details = verified ? "PASSED" : "FAILED";
        }
        else {
            res.success = false;
            res.details = errorMsg;
        }
    }
    else if (algorithm == "BWT+RLE") {
        ifstream inFile(inputPath, ios::binary);
        vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();
        res.originalSize = inputData.size();

        BWTResult bwtResult = BWT::encodeEfficient(inputData);
        vector<uint8_t> rleEncoded = RLE::encode(bwtResult.transformed, 1, 2);

        string bwtPath = encodedDir + "\\" + filename + "_bwt.bin";
        string rlePath = encodedDir + "\\" + filename + "_bwt.rle";
        string decodedPath = decodedDir + "\\" + filename;

        ofstream bwtFile(bwtPath, ios::binary);
        uint64_t primaryIdx = bwtResult.primaryIndex;
        uint64_t dataSize = bwtResult.transformed.size();
        bwtFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
        bwtFile.write(reinterpret_cast<const char*>(&primaryIdx), sizeof(primaryIdx));
        bwtFile.write(reinterpret_cast<const char*>(bwtResult.transformed.data()), dataSize);
        bwtFile.close();

        ofstream rleFile(rlePath, ios::binary);
        rleFile.write(reinterpret_cast<const char*>(rleEncoded.data()), rleEncoded.size());
        rleFile.close();

        res.processedSize = rleEncoded.size();
        res.ratio = (double)res.processedSize / res.originalSize * 100.0;

        string errorMsg;
```

```
        string tempDecode = encodedDir + "\\" + filename + "_temp.bin";
        RLE::decodeFile(rlePath, tempDecode, errorMsg);

        ifstream tempFile(tempDecode, ios::binary);
        uint64_t dataSize2;
        uint64_t primaryIdx2;
        tempFile.read(reinterpret_cast<char*>(&dataSize2), sizeof(dataSize2));
        tempFile.read(reinterpret_cast<char*>(&primaryIdx2), sizeof(primaryIdx2));
        vector<uint8_t> bwtData(dataSize2);
        tempFile.read(reinterpret_cast<char*>(bwtData.data()), dataSize2);
        tempFile.close();

        vector<uint8_t> decodedData = BWT::decode(bwtData, primaryIdx2);

        ofstream decFile(decodedPath, ios::binary);
        decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
        decFile.close();

        res.success = (decodedData.size() == inputData.size() &&
            memcmp(decodedData.data(), inputData.data(), inputData.size()) == 0);
        res.details = res.success ? "PASSED" : "FAILED";
    }
    else if (algorithm == "BWT+MTF+HA") {
        ifstream inFile(inputPath, ios::binary);
        vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();
        res.originalSize = inputData.size();

        BWTResult bwtResult = BWT::encodeEfficient(inputData);
        vector<uint8_t> mtfEncoded = MTF::encode(bwtResult.transformed);
        vector<uint8_t> haEncoded = HuffmanCanonical::encode(mtfEncoded);

        string bwtPath = encodedDir + "\\" + filename + "_bwt.bin";
        string mtfPath = encodedDir + "\\" + filename + "_mtf.bin";
        string haPath = encodedDir + "\\" + filename + ".ha";
        string decodedPath = decodedDir + "\\" + filename;

        ofstream bwtFile(bwtPath, ios::binary);
        uint64_t primaryIdx = bwtResult.primaryIndex;
        uint64_t dataSize = bwtResult.transformed.size();
        bwtFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
        bwtFile.write(reinterpret_cast<const char*>(&primaryIdx), sizeof(primaryIdx));
        bwtFile.write(reinterpret_cast<const char*>(bwtResult.transformed.data()), dataSize);
        bwtFile.close();

        ofstream mtfFile(mtfPath, ios::binary);
        mtfFile.write(reinterpret_cast<const char*>(mtfEncoded.data()), mtfEncoded.size());
        mtfFile.close();

        ofstream haFile(haPath, ios::binary);
        haFile.write(reinterpret_cast<const char*>(haEncoded.data()), haEncoded.size());
        haFile.close();

        res.processedSize = haEncoded.size();
        res.ratio = (double)res.processedSize / res.originalSize * 100.0;

        string errorMsg;
        string tempDecode = encodedDir + "\\" + filename + "_temp.bin";
        HuffmanCanonical::decodeFile(haPath, tempDecode, errorMsg);

        ifstream tempFile(tempDecode, ios::binary);
        vector<uint8_t> mtfData((istreambuf_iterator<char>(tempFile)), istreambuf_iterator<char>());
        tempFile.close();

        vector<uint8_t> mtfDecoded = MTF::decode(mtfData);

        ifstream bwtInFile(bwtPath, ios::binary);
        uint64_t dataSize2;
        uint64_t primaryIdx2;
        bwtInFile.read(reinterpret_cast<char*>(&dataSize2), sizeof(dataSize2));
        bwtInFile.read(reinterpret_cast<char*>(&primaryIdx2), sizeof(primaryIdx2));
        vector<uint8_t> bwtData(dataSize2);
        bwtInFile.read(reinterpret_cast<char*>(bwtData.data()), dataSize2);
        bwtInFile.close();

        vector<uint8_t> decodedData = BWT::decode(mtfDecoded, primaryIdx2);

        ofstream decFile(decodedPath, ios::binary);
        decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
        decFile.close();

        res.success = (decodedData.size() == inputData.size() &&
            memcmp(decodedData.data(), inputData.data(), inputData.size()) == 0);
        res.details = res.success ? "PASSED" : "FAILED";
    }
    else if (algorithm == "BWT+MTF+RLE+HA") {
        ifstream inFile(inputPath, ios::binary);
        vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();
        res.originalSize = inputData.size();
```

```
                BWTResult bwtResult = BWT::encodeEfficient(inputData);
                vector<uint8_t> mtfEncoded = MTF::encode(bwtResult.transformed);
                vector<uint8_t> rleEncoded = RLE::encode(mtfEncoded, 1, 2);
                vector<uint8_t> haEncoded = HuffmanCanonical::encode(rleEncoded);

                string bwtPath = encodedDir + "\\" + filename + "_bwt.bin";
                string mtfPath = encodedDir + "\\" + filename + "_mtf.bin";
                string rlePath = encodedDir + "\\" + filename + "_bwt.rle";
                string haPath = encodedDir + "\\" + filename + ".ha";
                string decodedPath = decodedDir + "\\" + filename;

                ofstream bwtFile(bwtPath, ios::binary);
                uint64_t primaryIdx = bwtResult.primaryIndex;
                uint64_t dataSize = bwtResult.transformed.size();
                bwtFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
                bwtFile.write(reinterpret_cast<const char*>(&primaryIdx), sizeof(primaryIdx));
                bwtFile.write(reinterpret_cast<const char*>(bwtResult.transformed.data()), dataSize);
                bwtFile.close();

                ofstream mtfFile(mtfPath, ios::binary);
                mtfFile.write(reinterpret_cast<const char*>(mtfEncoded.data()), mtfEncoded.size());
                mtfFile.close();

                ofstream rleFile(rlePath, ios::binary);
                rleFile.write(reinterpret_cast<const char*>(rleEncoded.data()), rleEncoded.size());
                rleFile.close();

                ofstream haFile(haPath, ios::binary);
                haFile.write(reinterpret_cast<const char*>(haEncoded.data()), haEncoded.size());
                haFile.close();

                res.processedSize = haEncoded.size();
                res.ratio = (double)res.processedSize / res.originalSize * 100.0;

                string errorMsg;
                string tempDecode1 = encodedDir + "\\" + filename + "_temp1.bin";
                string tempDecode2 = encodedDir + "\\" + filename + "_temp2.bin";
                HuffmanCanonical::decodeFile(haPath, tempDecode1, errorMsg);
                RLE::decodeFile(tempDecode1, tempDecode2, errorMsg);

                ifstream tempFile(tempDecode2, ios::binary);
                vector<uint8_t> mtfData((istreambuf_iterator<char>(tempFile)), istreambuf_iterator<char>());
                tempFile.close();

                vector<uint8_t> mtfDecoded = MTF::decode(mtfData);

                ifstream bwtInFile(bwtPath, ios::binary);
                uint64_t dataSize2;
                uint64_t primaryIdx2;
                bwtInFile.read(reinterpret_cast<char*>(&dataSize2), sizeof(dataSize2));
                bwtInFile.read(reinterpret_cast<char*>(&primaryIdx2), sizeof(primaryIdx2));
                vector<uint8_t> bwtData(dataSize2);
                bwtInFile.read(reinterpret_cast<char*>(bwtData.data()), dataSize2);
                bwtInFile.close();

                vector<uint8_t> decodedData = BWT::decode(mtfDecoded, primaryIdx2);

                ofstream decFile(decodedPath, ios::binary);
                decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
                decFile.close();

                res.success = (decodedData.size() == inputData.size() &&
                    memcmp(decodedData.data(), inputData.data(), inputData.size()) == 0);
                res.details = res.success ? "PASSED" : "FAILED";
            }
            else if (algorithm == "Entropy") {
                ifstream inFile(inputPath, ios::binary);
                vector<uint8_t> data((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
                inFile.close();
                res.originalSize = data.size();
                res.success = true;

                EntropyResult entropy = EntropyCalculator::calculate(data, 1);
                res.ratio = entropy.entropy;
                res.details = "Entropy: " + to_string(entropy.entropy) + " bits/byte";
            }

        }
        catch (const exception& e) {
            res.success = false;
            res.details = string("Exception: ") + e.what();
        }

        auto end = high_resolution_clock::now();
        res.timeMs = duration_cast<milliseconds>(end - start).count();

        results.push_back(res);
    }
}

void Analysis::runAllAnalyses(const string& inputDir,
```

```
    const string& encodedDir,
    const string& decodedDir) {
    cout << "\n========================================\n";
    cout << "Running All Analyses on All Files\n";
    cout << "========================================\n\n";

    vector<string> algorithms = {
        "RLE", "MTF", "Huffman", "Canonical Huffman", "LZ77", "LZSS", "LZ78", "LZW",
        "BWT+RLE", "BWT+MTF+HA", "BWT+MTF+RLE+HA", "Entropy"
    };

    vector<AlgorithmResult> allResults;

    for (const auto& algorithm : algorithms) {
        cout << "\n========================================\n";
        cout << "Testing: " << algorithm << "\n";
        cout << "========================================\n";

        vector<AlgorithmResult> results;
        runSingleAnalysis(algorithm, inputDir, encodedDir, decodedDir, results);

        for (const auto& res : results) {
            cout << "  " << res.filename << ": ";
            if (res.success) {
                cout << "PASSED";
                if (algorithm == "Entropy") {
                    cout << " (" << fixed << setprecision(4) << res.ratio << " bits/byte)";
                }
                else if (res.processedSize > 0 && res.originalSize > 0) {
                    cout << " (" << fixed << setprecision(2) << res.ratio << "%, "
                        << res.processedSize << "/" << res.originalSize << " bytes, "
                        << res.timeMs << " ms)";
                }
                else {
                    cout << " (" << res.timeMs << " ms)";
                }
            }
            else {
                cout << "FAILED - " << res.details;
            }
            cout << "\n";
        }

        allResults.insert(allResults.end(), results.begin(), results.end());
    }

    printAnalysisSummary(allResults);
}

void Analysis::printAnalysisSummary(const vector<AlgorithmResult>& results) {
    cout << "\n========================================\n";
    cout << "Analysis Summary\n";
    cout << "========================================\n\n";

    map<string, map<string, AlgorithmResult>> grouped;
    for (const auto& res : results) {
        grouped[res.algorithm][res.filename] = res;
    }

    cout << left << setw(25) << "Algorithm";
    vector<string> files = { "BW.raw", "Gray.raw", "RGB.raw", "enwik7.txt", "RusText.txt", "BIN.bin" };
    for (const auto& file : files) {
        cout << setw(15) << file.substr(0, 8);
    }
    cout << "\n";
    cout << string(25 + 15 * files.size(), '-') << "\n";

    for (const auto& algo : grouped) {
        cout << left << setw(25) << algo.first;
        for (const auto& file : files) {
            auto it = algo.second.find(file);
            if (it != algo.second.end() && it->second.success) {
                if (algo.first == "Entropy") {
                    cout << setw(15) << fixed << setprecision(2) << it->second.ratio;
                }
                else if (it->second.ratio > 0) {
                    cout << setw(15) << fixed << setprecision(1) << it->second.ratio << "%";
                }
                else {
                    cout << setw(15) << "PASS";
                }
            }
            else {
                cout << setw(15) << "FAIL";
            }
        }
        cout << "\n";
    }

    cout << "\n========================================\n";
    cout << "Success Rate by Algorithm\n";
```

```
    cout << "========================================\n\n";

    for (const auto& algo : grouped) {
        int success = 0;
        int total = 0;
        for (const auto& res : algo.second) {
            total++;
            if (res.second.success) success++;
        }
        cout << left << setw(25) << algo.first << ": " << success << "/" << total
            << " (" << fixed << setprecision(1) << (100.0 * success / total) << "%)\n";
    }

    cout << "\n========================================\n";
    cout << "Success Rate by File\n";
    cout << "========================================\n\n";

    map<string, int> fileSuccess, fileTotal;
    for (const auto& res : results) {
        fileTotal[res.filename]++;
        if (res.success) fileSuccess[res.filename]++;
    }

    for (const auto& file : files) {
        cout << left << setw(20) << file << ": " << fileSuccess[file] << "/" << fileTotal[file]
            << " (" << fixed << setprecision(1) << (100.0 * fileSuccess[file] / fileTotal[file]) << "%)\n";
    }

    cout << "\n";
}
```

bitstream.cpp

```
#include "bitstream.h"
#include <iostream>

using namespace std;

BitWriter::BitWriter() : currentByte(0), bitPos(0) {}

BitWriter::~BitWriter() {
    flush();
}

void BitWriter::writeBit(uint8_t bit) {
    currentByte |= (bit & 1) << (7 - bitPos);
    bitPos++;

    if (bitPos == 8) {
        buffer.push_back(currentByte);
        currentByte = 0;
        bitPos = 0;
    }
}

void BitWriter::writeBits(uint64_t value, int numBits) {
    for (int i = numBits - 1; i >= 0; i--) {
        writeBit((value >> i) & 1);
    }
}

void BitWriter::flush() {
    if (bitPos > 0) {
        buffer.push_back(currentByte);
        currentByte = 0;
        bitPos = 0;
    }
}

vector<uint8_t> BitWriter::getData() const {
    return buffer;
}

void BitWriter::clear() {
    buffer.clear();
    currentByte = 0;
    bitPos = 0;
}

size_t BitWriter::size() const {
    return buffer.size();
}

BitReader::BitReader(const vector<uint8_t>& input)
    : data(input), bytePos(0), bitPos(0) {
}

uint8_t BitReader::readBit() {
    if (bytePos >= data.size()) return 0;
```

```
    uint8_t bit = (data[bytePos] >> (7 - bitPos)) & 1;
    bitPos++;

    if (bitPos == 8) {
        bytePos++;
        bitPos = 0;
    }

    return bit;
}

uint64_t BitReader::readBits(int numBits) {
    uint64_t value = 0;
    for (int i = 0; i < numBits; i++) {
        value = (value << 1) | readBit();
    }
    return value;
}

bool BitReader::hasMore() const {
    return bytePos < data.size();
}

size_t BitReader::getPosition() const {
    return bytePos * 8 + bitPos;
}
```

bwt.cpp

```
#include "bwt.h"
#include "rle.h"
#include <vector>
#include <string>
#include <algorithm>
#include <fstream>
#include <iostream>
#include <iomanip>
#include <chrono>
#include <sys/stat.h>
#include "raw_convert.h"

using namespace std;
using namespace chrono;

bool fileExistsBWT(const string& path) {
    struct stat buffer;
    return (stat(path.c_str(), &buffer) == 0);
}

vector<vector<uint8_t>> BWT::buildRotationMatrix(const vector<uint8_t>& input) {
    size_t n = input.size();
    vector<vector<uint8_t>> matrix(n, vector<uint8_t>(n));

    for (size_t i = 0; i < n; i++) {
        for (size_t j = 0; j < n; j++) {
            matrix[i][j] = input[(i + j) % n];
        }
    }

    return matrix;
}

vector<size_t> BWT::getSortedIndices(const vector<vector<uint8_t>>& matrix) {
    size_t n = matrix.size();
    vector<size_t> indices(n);

    for (size_t i = 0; i < n; i++) {
        indices[i] = i;
    }

    sort(indices.begin(), indices.end(), [&matrix](size_t a, size_t b) {
        return matrix[a] < matrix[b];
        });

    return indices;
}

vector<uint8_t> BWT::getLastColumn(const vector<vector<uint8_t>>& matrix,
    const vector<size_t>& indices) {
    size_t n = matrix.size();
    vector<uint8_t> lastColumn(n);

    for (size_t i = 0; i < n; i++) {
        lastColumn[i] = matrix[indices[i]][n - 1];
    }

    return lastColumn;
}
```

```
BWTResult BWT::encode(const vector<uint8_t>& input) {
    BWTResult result;

    if (input.empty()) {
        result.description = "Empty input";
        return result;
    }

    size_t n = input.size();

    vector<vector<uint8_t>> matrix = buildRotationMatrix(input);

    vector<size_t> sortedIndices = getSortedIndices(matrix);

    result.transformed = getLastColumn(matrix, sortedIndices);

    for (size_t i = 0; i < n; i++) {
        if (sortedIndices[i] == 0) {
            result.primaryIndex = i;
            break;
        }
    }

    result.description = "BWT encoding successful";

    return result;
}

vector<uint8_t> BWT::reconstructUsingLFMapping(const vector<uint8_t>& lastColumn,
    size_t primaryIndex) {
    size_t n = lastColumn.size();
    vector<uint8_t> result(n);

    vector<pair<uint8_t, size_t>> column(n);
    for (size_t i = 0; i < n; i++) {
        column[i] = { lastColumn[i], i };
    }

    sort(column.begin(), column.end());

    size_t current = primaryIndex;
    for (size_t i = 0; i < n; i++) {
        result[i] = column[current].first;
        current = column[current].second;
    }

    return result;
}

vector<uint8_t> BWT::reconstructUsingPermutation(const vector<uint8_t>& lastColumn,
    size_t primaryIndex) {
    size_t n = lastColumn.size();
    vector<uint8_t> result(n);

    const int ALPHABET_SIZE = 256;
    vector<int> count(ALPHABET_SIZE, 0);

    for (size_t i = 0; i < n; i++) {
        count[lastColumn[i]]++;
    }

    vector<int> cumulativeCount(ALPHABET_SIZE, 0);
    for (int i = 1; i < ALPHABET_SIZE; i++) {
        cumulativeCount[i] = cumulativeCount[i - 1] + count[i - 1];
    }

    vector<size_t> firstColumnIndices(n);
    vector<int> tempCount = cumulativeCount;

    for (size_t i = 0; i < n; i++) {
        uint8_t symbol = lastColumn[i];
        firstColumnIndices[tempCount[symbol]] = i;
        tempCount[symbol]++;
    }

    vector<size_t> nextIndices(n);
    for (size_t i = 0; i < n; i++) {
        nextIndices[firstColumnIndices[i]] = i;
    }

    size_t current = primaryIndex;
    for (size_t i = 0; i < n; i++) {
        result[n - 1 - i] = lastColumn[current];
        current = nextIndices[current];
    }

    return result;
}

vector<uint8_t> BWT::decode(const vector<uint8_t>& input, size_t primaryIndex) {
    return reconstructUsingPermutation(input, primaryIndex);
```

```
}

vector<uint8_t> BWT::decodeWithCountingSort(const vector<uint8_t>& input, size_t primaryIndex) {
    return reconstructUsingPermutation(input, primaryIndex);
}

BWTBlockResult BWT::encodeBlocked(const vector<uint8_t>& input, size_t blockSize) {
    BWTBlockResult result;
    result.blockSize = blockSize;
    result.originalSize = input.size();

    if (input.empty()) {
        return result;
    }

    size_t numBlocks = (input.size() + blockSize - 1) / blockSize;
    result.transformed.reserve(input.size());
    result.primaryIndices.reserve(numBlocks);

    for (size_t blockIdx = 0; blockIdx < numBlocks; blockIdx++) {
        size_t start = blockIdx * blockSize;
        size_t end = min(start + blockSize, input.size());

        vector<uint8_t> block(input.begin() + start, input.begin() + end);

        if (block.size() < blockSize) {
            block.resize(blockSize, 0);
        }

        BWTResult bwtResult = encode(block);

        result.transformed.insert(result.transformed.end(),
            bwtResult.transformed.begin(),
            bwtResult.transformed.end());
        result.primaryIndices.push_back(bwtResult.primaryIndex);
    }

    return result;
}

vector<uint8_t> BWT::decodeBlocked(const BWTBlockResult& result) {
    vector<uint8_t> output;
    output.reserve(result.originalSize);

    size_t numBlocks = result.primaryIndices.size();
    size_t blockDataSize = result.blockSize;

    for (size_t blockIdx = 0; blockIdx < numBlocks; blockIdx++) {
        size_t start = blockIdx * blockDataSize;
        size_t end = min(start + blockDataSize, result.transformed.size());

        vector<uint8_t> blockData(result.transformed.begin() + start,
            result.transformed.begin() + end);

        vector<uint8_t> decodedBlock = decode(blockData, result.primaryIndices[blockIdx]);

        size_t remaining = result.originalSize - output.size();
        if (decodedBlock.size() > remaining) {
            decodedBlock.resize(remaining);
        }

        output.insert(output.end(), decodedBlock.begin(), decodedBlock.end());
    }

    return output;
}

bool BWT::encodeFileBlocked(const string& inputPath, const string& outputPath,
    size_t blockSize, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    BWTBlockResult bwtResult = encodeBlocked(inputData, blockSize);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    uint64_t magicNumber = 0x42575442;
    uint8_t version = 1;
    uint64_t originalSize = bwtResult.originalSize;
    uint64_t blockSize64 = bwtResult.blockSize;
    uint64_t numBlocks = bwtResult.primaryIndices.size();
```

```
    uint64_t dataSize = bwtResult.transformed.size();

    outFile.write(reinterpret_cast<const char*>(&magicNumber), sizeof(magicNumber));
    outFile.write(reinterpret_cast<const char*>(&version), sizeof(version));
    outFile.write(reinterpret_cast<const char*>(&originalSize), sizeof(originalSize));
    outFile.write(reinterpret_cast<const char*>(&blockSize64), sizeof(blockSize64));
    outFile.write(reinterpret_cast<const char*>(&numBlocks), sizeof(numBlocks));
    outFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));

    outFile.write(reinterpret_cast<const char*>(bwtResult.primaryIndices.data()),
        numBlocks * sizeof(size_t));
    outFile.write(reinterpret_cast<const char*>(bwtResult.transformed.data()), dataSize);
    outFile.close();

    return true;
}

bool BWT::decodeFileBlocked(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    uint64_t magicNumber;
    uint8_t version;
    uint64_t originalSize;
    uint64_t blockSize;
    uint64_t numBlocks;
    uint64_t dataSize;

    inFile.read(reinterpret_cast<char*>(&magicNumber), sizeof(magicNumber));
    if (magicNumber != 0x42575442) {
        errorMsg = "Invalid file format";
        return false;
    }

    inFile.read(reinterpret_cast<char*>(&version), sizeof(version));
    inFile.read(reinterpret_cast<char*>(&originalSize), sizeof(originalSize));
    inFile.read(reinterpret_cast<char*>(&blockSize), sizeof(blockSize));
    inFile.read(reinterpret_cast<char*>(&numBlocks), sizeof(numBlocks));
    inFile.read(reinterpret_cast<char*>(&dataSize), sizeof(dataSize));

    vector<size_t> primaryIndices(numBlocks);
    inFile.read(reinterpret_cast<char*>(primaryIndices.data()), numBlocks * sizeof(size_t));

    vector<uint8_t> transformedData(dataSize);
    inFile.read(reinterpret_cast<char*>(transformedData.data()), dataSize);
    inFile.close();

    BWTBlockResult bwtResult;
    bwtResult.originalSize = originalSize;
    bwtResult.blockSize = blockSize;
    bwtResult.primaryIndices = primaryIndices;
    bwtResult.transformed = transformedData;

    vector<uint8_t> decodedData = decodeBlocked(bwtResult);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

bool BWT::encodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    BWTResult result = encode(inputData);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    uint64_t primaryIdx = result.primaryIndex;
    uint64_t dataSize = result.transformed.size();
```

```
    outFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
    outFile.write(reinterpret_cast<const char*>(&primaryIdx), sizeof(primaryIdx));
    outFile.write(reinterpret_cast<const char*>(result.transformed.data()), result.transformed.size());
    outFile.close();

    return true;
}

bool BWT::decodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    uint64_t dataSize;
    uint64_t primaryIndex;

    inFile.read(reinterpret_cast<char*>(&dataSize), sizeof(dataSize));
    inFile.read(reinterpret_cast<char*>(&primaryIndex), sizeof(primaryIndex));

    vector<uint8_t> inputData(dataSize);
    inFile.read(reinterpret_cast<char*>(inputData.data()), dataSize);
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData, primaryIndex);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

bool BWT::decodeFileWithCountingSort(const string& inputPath, const string& outputPath, string& errorMsg) {
    return decodeFile(inputPath, outputPath, errorMsg);
}

bool BWT::verifyCycle(const string& inputPath, const string& encodedPath,
    const string& decodedPath, string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            return false;
        }
    }

    return true;
}

void BWT::testOnBanana() {
    cout << "\n========================================\n";
    cout << "BWT Transform Test on \"banana\"\n";
    cout << "========================================\n\n";

    string testStr = "banana";
    vector<uint8_t> input;
    for (char c : testStr) {
        input.push_back(static_cast<uint8_t>(c));
    }

    cout << "Original string: \"" << testStr << "\"\n";
    cout << "Original bytes: ";
    for (uint8_t b : input) {
```

```
        cout << "0x" << hex << setw(2) << setfill('0') << (int)b << " ";
    }
    cout << dec << "\n\n";

    BWTResult result = BWT::encode(input);

    cout << "BWT result:\n";
    cout << "  Last column: ";
    for (uint8_t b : result.transformed) {
        cout << (char)b;
    }
    cout << "\n";
    cout << "  Last column bytes: ";
    for (uint8_t b : result.transformed) {
        cout << "0x" << hex << setw(2) << setfill('0') << (int)b << " ";
    }
    cout << dec << "\n";
    cout << "  Primary index: " << result.primaryIndex << "\n\n";

    vector<uint8_t> decoded = BWT::decode(result.transformed, result.primaryIndex);

    string decodedStr;
    for (uint8_t b : decoded) {
        decodedStr += (char)b;
    }

    cout << "Decoded string: \"" << decodedStr << "\"\n";
    cout << "Decoded bytes: ";
    for (uint8_t b : decoded) {
        cout << "0x" << hex << setw(2) << setfill('0') << (int)b << " ";
    }
    cout << dec << "\n";
    cout << "Result: " << (testStr == decodedStr ? "PASSED" : "FAILED") << "\n\n";

    cout << "Applying RLE to BWT output:\n";
    vector<uint8_t> rleEncoded = RLE::encode(result.transformed, 1, 2);
    cout << "  BWT output size: " << result.transformed.size() << " bytes\n";
    cout << "  RLE compressed size: " << rleEncoded.size() << " bytes\n";
    cout << "  Compression ratio: " << fixed << setprecision(2)
        << (double)rleEncoded.size() / result.transformed.size() * 100.0 << "% of BWT output\n";
    cout << "  Overall compression (BWT+RLE): " << fixed << setprecision(2)
        << (double)rleEncoded.size() / input.size() * 100.0 << "% of original\n\n";
}

void BWT::testBWTAndRLE(const string& inputDir, const vector<string>& files) {
    size_t blockSize = 4096;

    for (const auto& filename : files) {
        string filePath = inputDir + "\\" + filename;

        if (!fileExistsBWT(filePath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "File: " << filename << "\n";
        cout << "----------------------------------------\n";

        ifstream inFile(filePath, ios::binary);
        if (!inFile.is_open()) {
            cout << "Error: Cannot open file\n\n";
            continue;
        }

        vector<uint8_t> originalData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();

        cout << "Original size: " << originalData.size() << " bytes\n";
        cout << "Block size: " << blockSize << " bytes\n\n";

        BWTBlockResult bwtResult = encodeBlocked(originalData, blockSize);

        cout << "BWT transformed size: " << bwtResult.transformed.size() << " bytes\n";
        cout << "Number of blocks: " << bwtResult.primaryIndices.size() << "\n";

        vector<uint8_t> rleEncoded = RLE::encode(bwtResult.transformed, 1, 2);

        size_t headerSize = sizeof(uint64_t) * 6 + bwtResult.primaryIndices.size() * sizeof(size_t);
        size_t totalSize = headerSize + rleEncoded.size();

        double ratio = (double)totalSize / originalData.size() * 100.0;
        double factor = (double)originalData.size() / totalSize;

        cout << "BWT+RLE compressed size: " << totalSize << " bytes\n";
        cout << "Compression ratio: " << fixed << setprecision(2) << ratio << "% of original\n";
        cout << "Compression factor: " << fixed << setprecision(2) << factor << ":1\n\n";

        string tempFile = inputDir + "\\temp_bwt_rle_" + filename + ".bin";
        ofstream tempOut(tempFile, ios::binary);
        if (tempOut.is_open()) {
            uint64_t magicNumber = 0x42575452;
```

```
            uint8_t version = 1;
            uint64_t originalSize = bwtResult.originalSize;
            uint64_t blockSize64 = bwtResult.blockSize;
            uint64_t numBlocks = bwtResult.primaryIndices.size();
            uint64_t rleSize = rleEncoded.size();

            tempOut.write(reinterpret_cast<const char*>(&magicNumber), sizeof(magicNumber));
            tempOut.write(reinterpret_cast<const char*>(&version), sizeof(version));
            tempOut.write(reinterpret_cast<const char*>(&originalSize), sizeof(originalSize));
            tempOut.write(reinterpret_cast<const char*>(&blockSize64), sizeof(blockSize64));
            tempOut.write(reinterpret_cast<const char*>(&numBlocks), sizeof(numBlocks));
            tempOut.write(reinterpret_cast<const char*>(&rleSize), sizeof(rleSize));
            tempOut.write(reinterpret_cast<const char*>(bwtResult.primaryIndices.data()),
                numBlocks * sizeof(size_t));
            tempOut.write(reinterpret_cast<const char*>(rleEncoded.data()), rleSize);
            tempOut.close();
        }

        ifstream tempIn(tempFile, ios::binary);
        if (tempIn.is_open()) {
            uint64_t magic, origSize, blockSize64, numBlocks, rleSizeVal;
            uint8_t version;

            tempIn.read(reinterpret_cast<char*>(&magic), sizeof(magic));
            tempIn.read(reinterpret_cast<char*>(&version), sizeof(version));
            tempIn.read(reinterpret_cast<char*>(&origSize), sizeof(origSize));
            tempIn.read(reinterpret_cast<char*>(&blockSize64), sizeof(blockSize64));
            tempIn.read(reinterpret_cast<char*>(&numBlocks), sizeof(numBlocks));
            tempIn.read(reinterpret_cast<char*>(&rleSizeVal), sizeof(rleSizeVal));

            vector<size_t> primIndices(numBlocks);
            tempIn.read(reinterpret_cast<char*>(primIndices.data()), numBlocks * sizeof(size_t));

            vector<uint8_t> rleData(rleSizeVal);
            tempIn.read(reinterpret_cast<char*>(rleData.data()), rleSizeVal);
            tempIn.close();

            vector<uint8_t> bwtDecoded = RLE::decode(rleData, 1, 2);

            BWTBlockResult decodeResult;
            decodeResult.originalSize = origSize;
            decodeResult.blockSize = blockSize64;
            decodeResult.primaryIndices = primIndices;
            decodeResult.transformed = bwtDecoded;

            vector<uint8_t> finalDecoded = decodeBlocked(decodeResult);

            if (originalData.size() == finalDecoded.size() &&
                memcmp(originalData.data(), finalDecoded.data(), originalData.size()) == 0) {
                cout << "Verification: PASSED\n";
            }
            else {
                cout << "Verification: FAILED\n";
            }

            remove(tempFile.c_str());
        }

        cout << "\n";
    }
}

vector<uint8_t> BWT::suffixArrayToLastColumn(
    const vector<uint8_t>& input,
    const vector<int32_t>& suffixArray) {

    vector<uint8_t> lastColumn;

    if (input.empty() || suffixArray.empty()) {
        return lastColumn;
    }

    size_t n = input.size();
    lastColumn.reserve(n);

    for (size_t i = 0; i < n; i++) {
        int32_t startPos = suffixArray[i];

        int32_t lastPos = (startPos == 0) ? static_cast<int32_t>(n - 1) : startPos - 1;

        lastColumn.push_back(input[lastPos]);
    }

    return lastColumn;
}

vector<int32_t> BWT::buildSuffixArrayDoubling(const vector<uint8_t>& input) {
    size_t n = input.size();

    if (n == 0) return vector<int32_t>();
    if (n == 1) return vector<int32_t>{0};
```

```
    vector<uint8_t> extended(2 * n);
    for (size_t i = 0; i < n; i++) {
        extended[i] = input[i];
        extended[i + n] = input[i];
    }

    vector<int32_t> sa(2 * n);
    vector<int32_t> rank(2 * n);
    vector<int32_t> newRank(2 * n);

    for (size_t i = 0; i < 2 * n; i++) {
        sa[i] = static_cast<int32_t>(i);
        rank[i] = extended[i];
    }

    sort(sa.begin(), sa.end(), [&extended](int32_t a, int32_t b) {
        return extended[a] < extended[b];
        });

    int32_t r = 0;
    rank[sa[0]] = r;
    for (size_t i = 1; i < 2 * n; i++) {
        if (extended[sa[i]] != extended[sa[i - 1]]) {
            r++;
        }
        rank[sa[i]] = r;
    }

    for (int32_t k = 1; k < static_cast<int32_t>(2 * n); k *= 2) {
        auto cmp = [&](int32_t a, int32_t b) -> bool {
            if (rank[a] != rank[b]) return rank[a] < rank[b];
            int32_t ra = (a + k < static_cast<int32_t>(2 * n)) ? rank[a + k] : -1;
            int32_t rb = (b + k < static_cast<int32_t>(2 * n)) ? rank[b + k] : -1;
            return ra < rb;
            };

        sort(sa.begin(), sa.end(), cmp);

        newRank[sa[0]] = 0;
        for (size_t i = 1; i < 2 * n; i++) {
            newRank[sa[i]] = newRank[sa[i - 1]];
            if (cmp(sa[i - 1], sa[i])) {
                newRank[sa[i]]++;
            }
        }

        for (size_t i = 0; i < 2 * n; i++) {
            rank[i] = newRank[i];
        }

        if (rank[sa[2 * n - 1]] == static_cast<int32_t>(2 * n - 1)) {
            break;
        }
    }

    vector<int32_t> result;
    for (size_t i = 0; i < 2 * n && result.size() < n; i++) {
        if (sa[i] < static_cast<int32_t>(n)) {
            result.push_back(sa[i]);
        }
    }

    return result;
}

size_t BWT::findPrimaryIndex(const vector<int32_t>& suffixArray) {
    for (size_t i = 0; i < suffixArray.size(); i++) {
        if (suffixArray[i] == 0) {
            return i;
        }
    }
    return 0;
}

BWTResult BWT::encodeEfficient(const vector<uint8_t>& input) {
    BWTResult result;

    if (input.empty()) {
        result.description = "Empty input";
        return result;
    }

    vector<int32_t> suffixArray = buildSuffixArrayDoubling(input);

    result.transformed = suffixArrayToLastColumn(input, suffixArray);

    result.primaryIndex = findPrimaryIndex(suffixArray);

    result.description = "BWT encoding successful (efficient suffix array-based)";
```

```
    return result;
}

BWTBlockResult BWT::encodeEfficientBlocked(const vector<uint8_t>& input, size_t blockSize) {
    BWTBlockResult result;
    result.blockSize = blockSize;
    result.originalSize = input.size();

    if (input.empty()) {
        return result;
    }

    size_t numBlocks = (input.size() + blockSize - 1) / blockSize;
    result.transformed.reserve(input.size());
    result.primaryIndices.reserve(numBlocks);

    for (size_t blockIdx = 0; blockIdx < numBlocks; blockIdx++) {
        size_t start = blockIdx * blockSize;
        size_t end = min(start + blockSize, input.size());

        vector<uint8_t> block(input.begin() + start, input.begin() + end);

        if (block.size() < blockSize) {
            block.resize(blockSize, 0);
        }

        BWTResult bwtResult = encodeEfficient(block);

        result.transformed.insert(result.transformed.end(),
            bwtResult.transformed.begin(),
            bwtResult.transformed.end());
        result.primaryIndices.push_back(bwtResult.primaryIndex);
    }

    return result;
}

void BWT::testEfficientBWT(const string& inputDir, const vector<string>& files) {
    size_t blockSize = 4096;

    cout << "\n========================================\n";
    cout << "Efficient BWT Test (Suffix Array Based)\n";
    cout << "========================================\n\n";

    for (const auto& filename : files) {
        string filePath = inputDir + "\\" + filename;

        if (!fileExistsBWT(filePath)) {
            cout << "File not found: " << filename << " - skipping\n\n";
            continue;
        }

        cout << "File: " << filename << "\n";
        cout << "----------------------------------------\n";

        ifstream inFile(filePath, ios::binary);
        if (!inFile.is_open()) {
            cout << "Error: Cannot open file\n\n";
            continue;
        }

        vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
        inFile.close();

        cout << "Original size: " << inputData.size() << " bytes\n";
        cout << "Block size: " << blockSize << " bytes\n";

        auto start = chrono::high_resolution_clock::now();
        BWTBlockResult efficientResult = encodeEfficientBlocked(inputData, blockSize);
        auto end = chrono::high_resolution_clock::now();
        auto efficientTime = chrono::duration_cast<chrono::milliseconds>(end - start);

        cout << "Efficient BWT time: " << efficientTime.count() << " ms\n";
        cout << "Transformed size: " << efficientResult.transformed.size() << " bytes\n";
        cout << "Number of blocks: " << efficientResult.primaryIndices.size() << "\n";

        cout << "Verifying by decoding... ";
        vector<uint8_t> decoded = decodeBlocked(efficientResult);

        if (decoded.size() == inputData.size() &&
            memcmp(decoded.data(), inputData.data(), inputData.size()) == 0) {
            cout << "PASSED\n";
        }
        else {
            cout << "FAILED\n";
        }

        cout << "\n";
    }
}
```

compressors.cpp

```
#include "compressors.h"
#include "rle.h"
#include "huffman_canonical.h"
#include "bwt.h"
#include "mtf.h"
#include "lzss.h"
#include "lzw.h"
#include <iostream>
#include <fstream>
#include <chrono>
#include <cstring>
#include <functional>

using namespace std;
using namespace chrono;

CompressionResult Compressors::HA(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string encodedPath = outputPath + ".huf";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();
    string errorMsg;
    bool success = HuffmanCanonical::encodeFile(inputPath, encodedPath, errorMsg);
    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream encodedFile(encodedPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(encodedFile)), istreambuf_iterator<char>());
    encodedFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();
    success = HuffmanCanonical::decodeFile(encodedPath, decodedPath, errorMsg);
    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);
    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    result.verified = (origData.size() == decData.size() &&
        memcmp(origData.data(), decData.data(), origData.size()) == 0);

    return result;
}

CompressionResult Compressors::RLE(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string encodedPath = outputPath + ".rle";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();
```

```
    string errorMsg;
    bool success = RLE::encodeFile(inputPath, encodedPath, 1, 2, errorMsg);
    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream encodedFile(encodedPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(encodedFile)), istreambuf_iterator<char>());
    encodedFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();
    success = RLE::decodeFile(encodedPath, decodedPath, errorMsg);
    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);
    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    result.verified = (origData.size() == decData.size() &&
        memcmp(origData.data(), decData.data(), origData.size()) == 0);

    return result;
}

CompressionResult Compressors::BWTRLE(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string bwtPath = outputPath + "_bwt.bin";
    string rlePath = outputPath + ".rle";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();

    BWTResult bwtResult = BWT::encodeEfficient(inputData);

    ofstream bwtFile(bwtPath, ios::binary);
    uint64_t primaryIdx = bwtResult.primaryIndex;
    uint64_t dataSize = bwtResult.transformed.size();
    bwtFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
    bwtFile.write(reinterpret_cast<const char*>(&primaryIdx), sizeof(primaryIdx));
    bwtFile.write(reinterpret_cast<const char*>(bwtResult.transformed.data()), dataSize);
    bwtFile.close();

    string errorMsg;
    RLE::encodeFile(bwtPath, rlePath, 1, 2, errorMsg);

    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    ifstream rleFile(rlePath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(rleFile)), istreambuf_iterator<char>());
    rleFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();

    string tempDecode = outputPath + "_temp.bin";
    RLE::decodeFile(rlePath, tempDecode, errorMsg);

    ifstream tempFile(tempDecode, ios::binary);
    uint64_t dataSize2;
    uint64_t primaryIdx2;
    tempFile.read(reinterpret_cast<char*>(&dataSize2), sizeof(dataSize2));
    tempFile.read(reinterpret_cast<char*>(&primaryIdx2), sizeof(primaryIdx2));
    vector<uint8_t> bwtData(dataSize2);
```

```
    tempFile.read(reinterpret_cast<char*>(bwtData.data()), dataSize2);
    tempFile.close();

    vector<uint8_t> decodedData = BWT::decode(bwtData, primaryIdx2);

    ofstream decFile(decodedPath, ios::binary);
    decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    decFile.close();

    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    result.verified = (decodedData.size() == inputData.size() &&
        memcmp(decodedData.data(), inputData.data(), inputData.size()) == 0);

    return result;
}

CompressionResult Compressors::BWTMTFHA(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string bwtPath = outputPath + "_bwt.bin";
    string mtfPath = outputPath + "_mtf.bin";
    string haPath = outputPath + ".ha";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();

    BWTResult bwtResult = BWT::encodeEfficient(inputData);

    ofstream bwtFile(bwtPath, ios::binary);
    uint64_t primaryIdx = bwtResult.primaryIndex;
    uint64_t dataSize = bwtResult.transformed.size();
    bwtFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
    bwtFile.write(reinterpret_cast<const char*>(&primaryIdx), sizeof(primaryIdx));
    bwtFile.write(reinterpret_cast<const char*>(bwtResult.transformed.data()), dataSize);
    bwtFile.close();

    vector<uint8_t> mtfEncoded = MTF::encode(bwtResult.transformed);

    ofstream mtfFile(mtfPath, ios::binary);
    mtfFile.write(reinterpret_cast<const char*>(mtfEncoded.data()), mtfEncoded.size());
    mtfFile.close();

    string errorMsg;
    HuffmanCanonical::encodeFile(mtfPath, haPath, errorMsg);

    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    ifstream haFile(haPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(haFile)), istreambuf_iterator<char>());
    haFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();

    string tempDecode = outputPath + "_temp.bin";
    HuffmanCanonical::decodeFile(haPath, tempDecode, errorMsg);

    ifstream tempFile(tempDecode, ios::binary);
    vector<uint8_t> mtfData((istreambuf_iterator<char>(tempFile)), istreambuf_iterator<char>());
    tempFile.close();

    vector<uint8_t> mtfDecoded = MTF::decode(mtfData);

    ifstream bwtInFile(bwtPath, ios::binary);
    uint64_t dataSize2;
    uint64_t primaryIdx2;
    bwtInFile.read(reinterpret_cast<char*>(&dataSize2), sizeof(dataSize2));
    bwtInFile.read(reinterpret_cast<char*>(&primaryIdx2), sizeof(primaryIdx2));
    vector<uint8_t> bwtData(dataSize2);
    bwtInFile.read(reinterpret_cast<char*>(bwtData.data()), dataSize2);
    bwtInFile.close();

    vector<uint8_t> decodedData = BWT::decode(mtfDecoded, primaryIdx2);

    ofstream decFile(decodedPath, ios::binary);
    decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
```

```
    decFile.close();

    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    result.verified = (decodedData.size() == inputData.size() &&
        memcmp(decodedData.data(), inputData.data(), inputData.size()) == 0);

    return result;
}

CompressionResult Compressors::BWTMTFRLEHA(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string bwtPath = outputPath + "_bwt.bin";
    string mtfPath = outputPath + "_mtf.bin";
    string rlePath = outputPath + ".rle";
    string haPath = outputPath + ".ha";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();

    BWTResult bwtResult = BWT::encodeEfficient(inputData);

    ofstream bwtFile(bwtPath, ios::binary);
    uint64_t primaryIdx = bwtResult.primaryIndex;
    uint64_t dataSize = bwtResult.transformed.size();
    bwtFile.write(reinterpret_cast<const char*>(&dataSize), sizeof(dataSize));
    bwtFile.write(reinterpret_cast<const char*>(&primaryIdx), sizeof(primaryIdx));
    bwtFile.write(reinterpret_cast<const char*>(bwtResult.transformed.data()), dataSize);
    bwtFile.close();

    vector<uint8_t> mtfEncoded = MTF::encode(bwtResult.transformed);

    ofstream mtfFile(mtfPath, ios::binary);
    mtfFile.write(reinterpret_cast<const char*>(mtfEncoded.data()), mtfEncoded.size());
    mtfFile.close();

    string errorMsg;
    RLE::encodeFile(mtfPath, rlePath, 1, 2, errorMsg);
    HuffmanCanonical::encodeFile(rlePath, haPath, errorMsg);

    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    ifstream haFile(haPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(haFile)), istreambuf_iterator<char>());
    haFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();

    string tempDecode1 = outputPath + "_temp1.bin";
    string tempDecode2 = outputPath + "_temp2.bin";
    HuffmanCanonical::decodeFile(haPath, tempDecode1, errorMsg);
    RLE::decodeFile(tempDecode1, tempDecode2, errorMsg);

    ifstream tempFile(tempDecode2, ios::binary);
    vector<uint8_t> mtfData((istreambuf_iterator<char>(tempFile)), istreambuf_iterator<char>());
    tempFile.close();

    vector<uint8_t> mtfDecoded = MTF::decode(mtfData);

    ifstream bwtInFile(bwtPath, ios::binary);
    uint64_t dataSize2;
    uint64_t primaryIdx2;
    bwtInFile.read(reinterpret_cast<char*>(&dataSize2), sizeof(dataSize2));
    bwtInFile.read(reinterpret_cast<char*>(&primaryIdx2), sizeof(primaryIdx2));
    vector<uint8_t> bwtData(dataSize2);
    bwtInFile.read(reinterpret_cast<char*>(bwtData.data()), dataSize2);
    bwtInFile.close();

    vector<uint8_t> decodedData = BWT::decode(mtfDecoded, primaryIdx2);

    ofstream decFile(decodedPath, ios::binary);
    decFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    decFile.close();

    end = high_resolution_clock::now();
```

```
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    result.verified = (decodedData.size() == inputData.size() &&
        memcmp(decodedData.data(), inputData.data(), inputData.size()) == 0);

    return result;
}

CompressionResult Compressors::LZSS(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string encodedPath = outputPath + ".lzss";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();
    string errorMsg;
    bool success = LZSS::encodeFile(inputPath, encodedPath, 4096, 16, errorMsg);
    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream encodedFile(encodedPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(encodedFile)), istreambuf_iterator<char>());
    encodedFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();
    success = LZSS::decodeFile(encodedPath, decodedPath, errorMsg);
    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);
    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    result.verified = (origData.size() == decData.size() &&
        memcmp(origData.data(), decData.data(), origData.size()) == 0);

    return result;
}

CompressionResult Compressors::LZSSHA(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string lzssPath = outputPath + ".lzss";
    string haPath = outputPath + ".ha";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();
    string errorMsg;
    LZSS::encodeFile(inputPath, lzssPath, 4096, 16, errorMsg);
    HuffmanCanonical::encodeFile(lzssPath, haPath, errorMsg);
    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    ifstream haFile(haPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(haFile)), istreambuf_iterator<char>());
    haFile.close();
    result.compressedSize = encodedData.size();
```

```
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();

    string tempDecode = outputPath + "_temp.bin";
    HuffmanCanonical::decodeFile(haPath, tempDecode, errorMsg);
    LZSS::decodeFile(tempDecode, decodedPath, errorMsg);

    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);
    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    result.verified = (origData.size() == decData.size() &&
        memcmp(origData.data(), decData.data(), origData.size()) == 0);

    return result;
}

CompressionResult Compressors::LZW(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();

    string encodedPath = outputPath + ".lzw";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();
    string errorMsg;
    bool success = LZW::encodeFile(inputPath, encodedPath, 4096, errorMsg);
    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream encodedFile(encodedPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(encodedFile)), istreambuf_iterator<char>());
    encodedFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();
    success = LZW::decodeFile(encodedPath, decodedPath, errorMsg);
    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    if (!success) {
        result.errorMsg = errorMsg;
        return result;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);
    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    result.verified = (origData.size() == decData.size() &&
        memcmp(origData.data(), decData.data(), origData.size()) == 0);

    return result;
}

CompressionResult Compressors::LZWHA(const string& inputPath, const string& outputPath) {
    CompressionResult result;
    result.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        result.errorMsg = "Cannot open input file";
        return result;
    }
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();
    result.originalSize = inputData.size();
```

```
    string lzwPath = outputPath + ".lzw";
    string haPath = outputPath + ".ha";
    string decodedPath = outputPath + ".dec";

    auto start = high_resolution_clock::now();
    string errorMsg;
    LZW::encodeFile(inputPath, lzwPath, 4096, errorMsg);
    HuffmanCanonical::encodeFile(lzwPath, haPath, errorMsg);
    auto end = high_resolution_clock::now();
    result.encodeTimeMs = duration_cast<milliseconds>(end - start).count();

    ifstream haFile(haPath, ios::binary);
    vector<uint8_t> encodedData((istreambuf_iterator<char>(haFile)), istreambuf_iterator<char>());
    haFile.close();
    result.compressedSize = encodedData.size();
    result.compressionRatio = (double)result.compressedSize / result.originalSize * 100.0;
    result.compressionFactor = (double)result.originalSize / result.compressedSize;

    start = high_resolution_clock::now();

    string tempDecode = outputPath + "_temp.bin";
    HuffmanCanonical::decodeFile(haPath, tempDecode, errorMsg);
    LZW::decodeFile(tempDecode, decodedPath, errorMsg);

    end = high_resolution_clock::now();
    result.decodeTimeMs = duration_cast<milliseconds>(end - start).count();

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);
    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    result.verified = (origData.size() == decData.size() &&
        memcmp(origData.data(), decData.data(), origData.size()) == 0);

    return result;
}

void Compressors::printResult(const CompressionResult& result) {
    if (!result.errorMsg.empty()) {
        cout << "  Error: " << result.errorMsg << "\n";
        return;
    }

    cout << "  Original: " << result.originalSize << " bytes\n";
    cout << "  Compressed: " << result.compressedSize << " bytes\n";
    cout << "  Ratio: " << fixed << setprecision(2) << result.compressionRatio << "%\n";
    cout << "  Factor: " << fixed << setprecision(2) << result.compressionFactor << ":1\n";
    cout << "  Encode time: " << result.encodeTimeMs << " ms\n";
    cout << "  Decode time: " << result.decodeTimeMs << " ms\n";
    cout << "  Status: " << (result.verified ? "PASSED" : "FAILED") << "\n";
}

void Compressors::printSummary(const vector<CompressionResult>& results) {
    cout << "\n========================================\n";
    cout << "Compression Summary\n";
    cout << "========================================\n\n";

    for (const auto& result : results) {
        cout << result.filename << ":\n";
        printResult(result);
        cout << "\n";
    }
}
```

entropy.cpp

```
#include "entropy.h"
#include <cmath>
#include <map>
#include <vector>
#include <iostream>
#include <iomanip>
#include <algorithm>

using namespace std;

EntropyResult EntropyCalculator::calculate(const vector<uint8_t>& data, uint8_t Ms) {
    EntropyResult result;
    result.totalSymbols = 0;
    result.uniqueSymbols = 0;
    result.entropy = 0.0;

    if (data.empty() || Ms == 0) {
        return result;
    }

    map<vector<uint8_t>, uint64_t> frequencies;

    size_t numSymbols = data.size() / Ms;
```

```
    for (size_t i = 0; i < numSymbols; i++) {
        vector<uint8_t> symbol;
        for (uint8_t k = 0; k < Ms; k++) {
            symbol.push_back(data[i * Ms + k]);
        }
        frequencies[symbol]++;
        result.totalSymbols++;
    }

    if (data.size() % Ms != 0) {
        vector<uint8_t> lastSymbol;
        for (size_t i = numSymbols * Ms; i < data.size(); i++) {
            lastSymbol.push_back(data[i]);
        }
        while (lastSymbol.size() < Ms) {
            lastSymbol.push_back(0);
        }
        frequencies[lastSymbol]++;
        result.totalSymbols++;
    }

    result.frequencies = frequencies;
    result.uniqueSymbols = frequencies.size();

    result.entropy = calculateShannonEntropy(frequencies, result.totalSymbols);

    return result;
}

double EntropyCalculator::calculateShannonEntropy(const map<vector<uint8_t>, uint64_t>& frequencies,
    uint64_t totalSymbols) {
    if (totalSymbols == 0) return 0.0;

    double entropy = 0.0;

    for (const auto& pair : frequencies) {
        double probability = (double)pair.second / totalSymbols;
        entropy -= probability * log2(probability);
    }

    return entropy;
}
```

huffman.cpp

```
#include "huffman.h"
#include "bitstream.h"
#include <vector>
#include <map>
#include <queue>
#include <fstream>
#include <algorithm>
#include <memory>
#include <cmath>
#include <iomanip>
#include <iostream>

using namespace std;

HuffmanNode::HuffmanNode(uint8_t sym, uint64_t freq)
    : symbol(sym), frequency(freq), left(nullptr), right(nullptr) {
}

HuffmanNode::HuffmanNode(uint64_t freq, shared_ptr<HuffmanNode> l, shared_ptr<HuffmanNode> r)
    : symbol(0), frequency(freq), left(l), right(r) {
}

bool HuffmanNode::isLeaf() const {
    return left == nullptr && right == nullptr;
}

map<uint8_t, uint64_t> Huffman::calculateFrequencies(const vector<uint8_t>& data) {
    map<uint8_t, uint64_t> frequencies;
    for (uint8_t byte : data) {
        frequencies[byte]++;
    }
    return frequencies;
}

shared_ptr<HuffmanNode> Huffman::buildTree(const map<uint8_t, uint64_t>& frequencies) {
    auto cmp = [](const shared_ptr<HuffmanNode>& a, const shared_ptr<HuffmanNode>& b) {
        return a->frequency > b->frequency;
        };

    priority_queue<shared_ptr<HuffmanNode>, vector<shared_ptr<HuffmanNode>>, decltype(cmp)> pq(cmp);

    for (const auto& pair : frequencies) {
        pq.push(make_shared<HuffmanNode>(pair.first, pair.second));
```

```
    }

    if (pq.empty()) {
        return nullptr;
    }

    if (pq.size() == 1) {
        auto node = pq.top();
        pq.pop();
        return make_shared<HuffmanNode>(node->frequency, node, nullptr);
    }

    while (pq.size() > 1) {
        auto left = pq.top();
        pq.pop();
        auto right = pq.top();
        pq.pop();

        auto parent = make_shared<HuffmanNode>(left->frequency + right->frequency, left, right);
        pq.push(parent);
    }

    return pq.top();
}

void Huffman::generateCodes(shared_ptr<HuffmanNode> node, vector<bool>& currentCode, map<uint8_t, vector<bool>>& codes) {
    if (!node) return;

    if (node->isLeaf()) {
        codes[node->symbol] = currentCode;
        return;
    }

    if (node->left) {
        currentCode.push_back(0);
        generateCodes(node->left, currentCode, codes);
        currentCode.pop_back();
    }

    if (node->right) {
        currentCode.push_back(1);
        generateCodes(node->right, currentCode, codes);
        currentCode.pop_back();
    }
}

void Huffman::writeTreeToBitStream(shared_ptr<HuffmanNode> node, BitWriter& writer) {
    if (!node) return;

    if (node->isLeaf()) {
        writer.writeBit(1);
        writer.writeBits(node->symbol, 8);
    }
    else {
        writer.writeBit(0);
        writeTreeToBitStream(node->left, writer);
        writeTreeToBitStream(node->right, writer);
    }
}

shared_ptr<HuffmanNode> Huffman::readTreeFromBitStream(BitReader& reader) {
    if (!reader.hasMore()) return nullptr;

    uint8_t flag = reader.readBit();

    if (flag == 1) {
        uint8_t symbol = static_cast<uint8_t>(reader.readBits(8));
        return make_shared<HuffmanNode>(symbol, 0);
    }
    else {
        auto left = readTreeFromBitStream(reader);
        auto right = readTreeFromBitStream(reader);
        return make_shared<HuffmanNode>(0, left, right);
    }
}

size_t Huffman::calculateTreeSize(shared_ptr<HuffmanNode> node) {
    if (!node) return 0;

    size_t size = 0;
    if (node->isLeaf()) {
        size += 1 + 1;
    }
    else {
        size += 1;
        size += calculateTreeSize(node->left);
        size += calculateTreeSize(node->right);
    }
    return size;
}
```

```
uint32_t Huffman::calculateChecksum(const vector<uint8_t>& data) {
    uint32_t checksum = 0;
    for (uint8_t byte : data) {
        checksum = (checksum << 5) - checksum + byte;
    }
    return checksum;
}

vector<uint8_t> Huffman::encode(const vector<uint8_t>& input) {
    BitWriter writer;

    if (input.empty()) {
        return writer.getData();
    }

    map<uint8_t, uint64_t> frequencies = calculateFrequencies(input);

    shared_ptr<HuffmanNode> root = buildTree(frequencies);

    if (!root) {
        return writer.getData();
    }

    writeTreeToBitStream(root, writer);

    uint64_t originalSize = input.size();
    writer.writeBits(originalSize, 64);

    map<uint8_t, vector<bool>> codes;
    vector<bool> currentCode;
    generateCodes(root, currentCode, codes);

    for (uint8_t byte : input) {
        const vector<bool>& code = codes[byte];
        for (bool bit : code) {
            writer.writeBit(bit ? 1 : 0);
        }
    }

    writer.flush();
    return writer.getData();
}

vector<uint8_t> Huffman::decode(const vector<uint8_t>& input) {
    BitReader reader(input);
    vector<uint8_t> output;

    if (input.empty()) {
        return output;
    }

    shared_ptr<HuffmanNode> root = readTreeFromBitStream(reader);

    if (!root) {
        return output;
    }

    if (!reader.hasMore()) {
        return output;
    }

    uint64_t originalSize = reader.readBits(64);

    for (uint64_t i = 0; i < originalSize; i++) {
        shared_ptr<HuffmanNode> current = root;

        while (current && !current->isLeaf()) {
            if (!reader.hasMore()) {
                return output;
            }
            uint8_t bit = reader.readBit();
            if (bit == 0) {
                current = current->left;
            }
            else {
                current = current->right;
            }
        }

        if (current && current->isLeaf()) {
            output.push_back(current->symbol);
        }
        else {
            return output;
        }
    }

    return output;
}

bool Huffman::encodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
```

```
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> encodedData = encode(inputData);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool Huffman::decodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}



bool Huffman::encodeFileWithStats(const string& inputPath, const string& outputPath,
    CompressionStatsHuffman& stats, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    map<uint8_t, uint64_t> frequencies = calculateFrequencies(inputData);
    shared_ptr<HuffmanNode> root = buildTree(frequencies);

    if (!root) {
        errorMsg = "Failed to build Huffman tree";
        return false;
    }

    stats.treeSize = calculateTreeSize(root);

    BitWriter writer;
    writeTreeToBitStream(root, writer);
    stats.metadataSize = writer.getData().size();

    writer.writeBits(stats.originalSize, 64);

    map<uint8_t, vector<bool>> codes;
    vector<bool> currentCode;
    generateCodes(root, currentCode, codes);

    for (uint8_t byte : inputData) {
        const vector<bool>& code = codes[byte];
        for (bool bit : code) {
            writer.writeBit(bit ? 1 : 0);
        }
    }

    writer.flush();
```

```
    vector<uint8_t> encodedData = writer.getData();

    stats.encodedSize = encodedData.size();
    stats.encodedDataSize = stats.encodedSize - stats.metadataSize - 8;
    stats.compressionRatio = (double)stats.encodedSize / stats.originalSize * 100.0;
    stats.compressionFactor = (double)stats.originalSize / stats.encodedSize;
    stats.metadataOverhead = (double)stats.metadataSize / stats.encodedSize * 100.0;

    double entropy = 0.0;
    for (const auto& pair : frequencies) {
        double prob = (double)pair.second / stats.originalSize;
        entropy -= prob * log2(prob);
    }

    stats.efficiency = (entropy * stats.originalSize / 8.0) / stats.encodedSize * 100.0;

    uint32_t checksum = calculateChecksum(encodedData);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    HuffmanFileHeader header;
    header.magicNumber = 0x4846464D;
    header.version = 1;
    header.originalSize = stats.originalSize;
    header.encodedDataSize = encodedData.size();
    header.checksum = checksum;

    outFile.write(reinterpret_cast<const char*>(&header.magicNumber), sizeof(header.magicNumber));
    outFile.write(reinterpret_cast<const char*>(&header.version), sizeof(header.version));
    outFile.write(reinterpret_cast<const char*>(&header.originalSize), sizeof(header.originalSize));
    outFile.write(reinterpret_cast<const char*>(&header.encodedDataSize), sizeof(header.encodedDataSize));
    outFile.write(reinterpret_cast<const char*>(&header.checksum), sizeof(header.checksum));
    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool Huffman::decodeFileWithVerify(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    HuffmanFileHeader header;
    inFile.read(reinterpret_cast<char*>(&header.magicNumber), sizeof(header.magicNumber));
    if (header.magicNumber != 0x4846464D) {
        errorMsg = "Invalid file format: wrong magic number";
        return false;
    }

    inFile.read(reinterpret_cast<char*>(&header.version), sizeof(header.version));
    if (header.version != 1) {
        errorMsg = "Unsupported file version";
        return false;
    }

    inFile.read(reinterpret_cast<char*>(&header.originalSize), sizeof(header.originalSize));
    inFile.read(reinterpret_cast<char*>(&header.encodedDataSize), sizeof(header.encodedDataSize));
    inFile.read(reinterpret_cast<char*>(&header.checksum), sizeof(header.checksum));

    vector<uint8_t> encodedData(header.encodedDataSize);
    inFile.read(reinterpret_cast<char*>(encodedData.data()), header.encodedDataSize);
    inFile.close();

    uint32_t checksum = calculateChecksum(encodedData);
    if (checksum != header.checksum) {
        errorMsg = "Checksum verification failed: file may be corrupted";
        return false;
    }

    vector<uint8_t> decodedData = decode(encodedData);

    if (decodedData.size() != header.originalSize) {
        errorMsg = "Decoded size mismatch: expected " + to_string(header.originalSize) +
            ", got " + to_string(decodedData.size());
        return false;
    }

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
```

```
    outFile.close();

    return true;
}



bool Huffman::verifyCycle(const string& inputPath, const string& encodedPath,
    const string& decodedPath, string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            return false;
        }
    }

    return true;
}

double Huffman::getCompressionRatio(size_t originalSize, size_t encodedSize) {
    if (originalSize == 0) return 0.0;
    return (double)encodedSize / originalSize * 100.0;
}

CompressionStatsHuffman Huffman::analyzeFile(const string& inputPath, string& errorMsg) {
    CompressionStatsHuffman stats;
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);
    stats.originalSize = 0;
    stats.encodedSize = 0;
    stats.metadataSize = 0;
    stats.treeSize = 0;
    stats.encodedDataSize = 0;
    stats.compressionRatio = 0.0;
    stats.compressionFactor = 0.0;
    stats.metadataOverhead = 0.0;
    stats.efficiency = 0.0;

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open file: " + inputPath;
        return stats;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();

    map<uint8_t, uint64_t> frequencies = calculateFrequencies(inputData);
    shared_ptr<HuffmanNode> root = buildTree(frequencies);

    if (!root) {
        errorMsg = "Failed to build Huffman tree";
        return stats;
    }

    stats.treeSize = calculateTreeSize(root);

    BitWriter writer;
    writeTreeToBitStream(root, writer);
    stats.metadataSize = writer.getData().size();

    writer.writeBits(stats.originalSize, 64);

    map<uint8_t, vector<bool>> codes;
    vector<bool> currentCode;
    generateCodes(root, currentCode, codes);

    for (uint8_t byte : inputData) {
```

```
        const vector<bool>& code = codes[byte];
        for (bool bit : code) {
            writer.writeBit(bit ? 1 : 0);
        }
    }

    writer.flush();
    vector<uint8_t> encodedData = writer.getData();

    stats.encodedSize = encodedData.size();
    stats.encodedDataSize = stats.encodedSize - stats.metadataSize - 8;
    stats.compressionRatio = (double)stats.encodedSize / stats.originalSize * 100.0;
    stats.compressionFactor = (double)stats.originalSize / stats.encodedSize;
    stats.metadataOverhead = (double)stats.metadataSize / stats.encodedSize * 100.0;

    double entropy = 0.0;
    for (const auto& pair : frequencies) {
        double prob = (double)pair.second / stats.originalSize;
        entropy -= prob * log2(prob);
    }

    stats.efficiency = (entropy * stats.originalSize / 8.0) / stats.encodedSize * 100.0;

    return stats;
}

void Huffman::printStats(const CompressionStatsHuffman& stats) {
    cout << "File: " << stats.filename << "\n";
    cout << "----------------------------------------\n";
    cout << "Original size: " << stats.originalSize << " bytes\n";
    cout << "Encoded size:  " << stats.encodedSize << " bytes\n";
    cout << "Metadata size: " << stats.metadataSize << " bytes\n";
    cout << "  - Tree structure: " << stats.treeSize << " bytes\n";
    cout << "  - Original size: 8 bytes\n";
    cout << "  - Header: " << (stats.metadataSize - stats.treeSize - 8) << " bytes\n";
    cout << "Compression ratio: " << fixed << setprecision(2) << stats.compressionRatio << "% of original\n";
    cout << "Compression factor: " << fixed << setprecision(2) << stats.compressionFactor << ":1\n";
    cout << "Metadata overhead: " << fixed << setprecision(2) << stats.metadataOverhead << "% of encoded file\n";
    cout << "Huffman efficiency: " << fixed << setprecision(2) << stats.efficiency << "% of entropy bound\n";
}
```

huffman_canonical.cpp

```
#include "huffman_canonical.h"
#include "bitstream.h"
#include <iostream>
#include <fstream>
#include <algorithm>
#include <cstring>
#include <queue>
#include <cmath>
#include <functional>

using namespace std;

const uint32_t HUFFMAN_CANONICAL_MAGIC = 0x4843464D;
const uint8_t HUFFMAN_CANONICAL_VERSION = 1;

struct HuffmanNodeCanonical {
    uint8_t symbol;
    uint64_t frequency;
    shared_ptr<HuffmanNodeCanonical> left;
    shared_ptr<HuffmanNodeCanonical> right;

    HuffmanNodeCanonical(uint8_t sym, uint64_t freq) : symbol(sym), frequency(freq), left(nullptr), right(nullptr) {}
    HuffmanNodeCanonical(uint64_t freq, shared_ptr<HuffmanNodeCanonical> l, shared_ptr<HuffmanNodeCanonical> r)
        : symbol(0), frequency(freq), left(l), right(r) {
    }
    bool isLeaf() const { return left == nullptr && right == nullptr; }
};

uint32_t HuffmanCanonical::calculateChecksum(const vector<uint8_t>& data) {
    uint32_t checksum = 0;
    for (uint8_t byte : data) {
        checksum = (checksum << 5) - checksum + byte;
    }
    return checksum;
}

map<uint8_t, uint64_t> HuffmanCanonical::calculateFrequencies(const vector<uint8_t>& data) {
    map<uint8_t, uint64_t> frequencies;
    for (uint8_t byte : data) {
        frequencies[byte]++;
    }
    return frequencies;
}

vector<uint8_t> HuffmanCanonical::getCodeLengths(const map<uint8_t, uint64_t>& frequencies) {
    if (frequencies.empty()) return vector<uint8_t>(256, 0);
```

```
    auto cmp = [](const shared_ptr<HuffmanNodeCanonical>& a, const shared_ptr<HuffmanNodeCanonical>& b) {
        return a->frequency > b->frequency;
        };

    priority_queue<shared_ptr<HuffmanNodeCanonical>, vector<shared_ptr<HuffmanNodeCanonical>>, decltype(cmp)> pq(cmp);

    for (const auto& pair : frequencies) {
        pq.push(make_shared<HuffmanNodeCanonical>(pair.first, pair.second));
    }

    if (pq.size() == 1) {
        auto node = pq.top();
        vector<uint8_t> codeLengths(256, 0);
        codeLengths[node->symbol] = 1;
        return codeLengths;
    }

    while (pq.size() > 1) {
        auto left = pq.top(); pq.pop();
        auto right = pq.top(); pq.pop();
        auto parent = make_shared<HuffmanNodeCanonical>(left->frequency + right->frequency, left, right);
        pq.push(parent);
    }

    auto root = pq.top();
    vector<uint8_t> codeLengths(256, 0);

    function<void(shared_ptr<HuffmanNodeCanonical>, int)> traverse;
    traverse = [&](shared_ptr<HuffmanNodeCanonical> node, int depth) {
        if (!node) return;
        if (node->isLeaf()) {
            codeLengths[node->symbol] = depth;
        }
        else {
            traverse(node->left, depth + 1);
            traverse(node->right, depth + 1);
        }
        };

    traverse(root, 0);

    return codeLengths;
}

map<uint8_t, CanonicalCode> HuffmanCanonical::buildCanonicalCodes(const vector<uint8_t>& codeLengths) {
    map<uint8_t, CanonicalCode> codes;

    vector<pair<uint8_t, uint8_t>> symbolsWithLength;
    for (int i = 0; i < 256; i++) {
        if (codeLengths[i] > 0) {
            symbolsWithLength.push_back({ static_cast<uint8_t>(i), codeLengths[i] });
        }
    }

    sort(symbolsWithLength.begin(), symbolsWithLength.end(),
        [](const pair<uint8_t, uint8_t>& a, const pair<uint8_t, uint8_t>& b) {
            if (a.second != b.second) return a.second < b.second;
            return a.first < b.first;
        });

    uint32_t currentCode = 0;
    uint8_t prevLength = 0;

    for (const auto& symLen : symbolsWithLength) {
        uint8_t symbol = symLen.first;
        uint8_t length = symLen.second;

        if (length > prevLength) {
            currentCode <<= (length - prevLength);
        }

        CanonicalCode code;
        code.code = currentCode;
        code.length = length;
        codes[symbol] = code;

        currentCode++;
        prevLength = length;
    }

    return codes;
}

vector<uint8_t> HuffmanCanonical::encodeCanonical(const vector<uint8_t>& input) {
    BitWriter writer;

    if (input.empty()) return writer.getData();

    map<uint8_t, uint64_t> frequencies = calculateFrequencies(input);
    vector<uint8_t> codeLengths = getCodeLengths(frequencies);
    map<uint8_t, CanonicalCode> codes = buildCanonicalCodes(codeLengths);
```

```
    writer.writeBits(HUFFMAN_CANONICAL_MAGIC, 32);
    writer.writeBits(HUFFMAN_CANONICAL_VERSION, 8);
    writer.writeBits(input.size(), 64);

    uint16_t numSymbols = 0;
    for (int i = 0; i < 256; i++) {
        if (codeLengths[i] > 0) numSymbols++;
    }
    writer.writeBits(numSymbols, 16);

    for (int i = 0; i < 256; i++) {
        if (codeLengths[i] > 0) {
            writer.writeBits(i, 8);
            writer.writeBits(codeLengths[i], 8);
        }
    }

    for (uint8_t byte : input) {
        const CanonicalCode& code = codes[byte];
        writer.writeBits(code.code, code.length);
    }

    writer.flush();
    return writer.getData();
}

vector<uint8_t> HuffmanCanonical::decodeCanonical(const vector<uint8_t>& input, size_t originalSize, const vector<uint8_t>&
codeLengths) {
    BitReader reader(input);
    vector<uint8_t> output;
    output.reserve(originalSize);

    vector<pair<uint8_t, uint8_t>> symbolsWithLength;
    for (int i = 0; i < 256; i++) {
        if (codeLengths[i] > 0) {
            symbolsWithLength.push_back({ static_cast<uint8_t>(i), codeLengths[i] });
        }
    }

    sort(symbolsWithLength.begin(), symbolsWithLength.end(),
        [](const pair<uint8_t, uint8_t>& a, const pair<uint8_t, uint8_t>& b) {
            if (a.second != b.second) return a.second < b.second;
            return a.first < b.first;
        });

    vector<pair<uint32_t, uint8_t>> codeToSymbol;
    uint32_t currentCode = 0;
    uint8_t prevLength = 0;

    for (const auto& symLen : symbolsWithLength) {
        uint8_t symbol = symLen.first;
        uint8_t length = symLen.second;

        if (length > prevLength) {
            currentCode <<= (length - prevLength);
        }

        codeToSymbol.push_back({ currentCode, symbol });

        currentCode++;
        prevLength = length;
    }

    while (output.size() < originalSize && reader.hasMore()) {
        uint32_t currentValue = 0;
        int bitsRead = 0;
        bool found = false;

        while (!found && reader.hasMore()) {
            currentValue = (currentValue << 1) | reader.readBit();
            bitsRead++;

            for (const auto& pair : codeToSymbol) {
                if (pair.first == currentValue) {
                    uint8_t length = 0;
                    for (const auto& sl : symbolsWithLength) {
                        if (sl.first == pair.second) {
                            length = sl.second;
                            break;
                        }
                    }
                    if (bitsRead == length) {
                        output.push_back(pair.second);
                        found = true;
                        break;
                    }
                }
            }
        }
```

```
        if (!found) break;
    }

    return output;
}

vector<uint8_t> HuffmanCanonical::encode(const vector<uint8_t>& input) {
    if (input.empty()) {
        vector<uint8_t> result(sizeof(CanonicalHuffmanHeader));
        CanonicalHuffmanHeader header;
        header.magicNumber = HUFFMAN_CANONICAL_MAGIC;
        header.version = HUFFMAN_CANONICAL_VERSION;
        header.originalSize = 0;
        header.compressedSize = 0;
        header.checksum = 0;
        header.numSymbols = 0;
        memcpy(result.data(), &header, sizeof(CanonicalHuffmanHeader));
        return result;
    }

    vector<uint8_t> compressedData = encodeCanonical(input);

    CanonicalHuffmanHeader header;
    header.magicNumber = HUFFMAN_CANONICAL_MAGIC;
    header.version = HUFFMAN_CANONICAL_VERSION;
    header.originalSize = input.size();
    header.compressedSize = compressedData.size();
    header.checksum = calculateChecksum(compressedData);

    uint16_t numSymbols = 0;
    map<uint8_t, uint64_t> freqs = calculateFrequencies(input);
    for (const auto& p : freqs) {
        if (p.second > 0) numSymbols++;
    }
    header.numSymbols = numSymbols;

    vector<uint8_t> result(sizeof(CanonicalHuffmanHeader) + compressedData.size());
    memcpy(result.data(), &header, sizeof(CanonicalHuffmanHeader));
    memcpy(result.data() + sizeof(CanonicalHuffmanHeader), compressedData.data(), compressedData.size());

    return result;
}

vector<uint8_t> HuffmanCanonical::decode(const vector<uint8_t>& input) {
    if (input.size() < sizeof(CanonicalHuffmanHeader)) {
        return vector<uint8_t>();
    }

    CanonicalHuffmanHeader header;
    memcpy(&header, input.data(), sizeof(CanonicalHuffmanHeader));

    if (header.magicNumber != HUFFMAN_CANONICAL_MAGIC) {
        return vector<uint8_t>();
    }

    if (header.version != HUFFMAN_CANONICAL_VERSION) {
        return vector<uint8_t>();
    }

    if (header.originalSize == 0) {
        return vector<uint8_t>();
    }

    if (input.size() < sizeof(CanonicalHuffmanHeader) + header.compressedSize) {
        return vector<uint8_t>();
    }

    vector<uint8_t> compressedData(input.begin() + sizeof(CanonicalHuffmanHeader),
        input.begin() + sizeof(CanonicalHuffmanHeader) + header.compressedSize);

    uint32_t checksum = calculateChecksum(compressedData);
    if (checksum != header.checksum) {
        return vector<uint8_t>();
    }

    BitReader reader(compressedData);

    uint32_t magic = static_cast<uint32_t>(reader.readBits(32));
    uint8_t version = static_cast<uint8_t>(reader.readBits(8));
    uint64_t originalSize = reader.readBits(64);
    uint16_t numSymbols = static_cast<uint16_t>(reader.readBits(16));

    vector<uint8_t> codeLengths(256, 0);
    for (int i = 0; i < numSymbols; i++) {
        uint8_t symbol = static_cast<uint8_t>(reader.readBits(8));
        uint8_t length = static_cast<uint8_t>(reader.readBits(8));
        codeLengths[symbol] = length;
    }

    vector<uint8_t> remainingData;
    while (reader.hasMore()) {
```

```
        remainingData.push_back(static_cast<uint8_t>(reader.readBits(8)));
    }

    return decodeCanonical(remainingData, originalSize, codeLengths);
}

bool HuffmanCanonical::encodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> encodedData = encode(inputData);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool HuffmanCanonical::decodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)), istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData);

    if (decodedData.empty()) {
        errorMsg = "Failed to decode data";
        return false;
    }

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

bool HuffmanCanonical::verifyCycle(const string& inputPath, const string& encodedPath, const string& decodedPath, string&
errorMsg) {
    if (!encodeFile(inputPath, encodedPath, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)), istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)), istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            return false;
        }
    }

    return true;
```

```
}
```

lz77.cpp

```
#include "lz77.h"
#include "bitstream.h"
#include <iostream>
#include <fstream>
#include <algorithm>
#include <cstring>
#include <iomanip>
#include <chrono>

using namespace std;


const uint32_t LZ77_MAGIC = 0x4C5A3737;
const uint8_t LZ77_VERSION = 1;

uint32_t LZ77::calculateChecksum(const vector<uint8_t>& data) {
    uint32_t checksum = 0;
    for (uint8_t byte : data) {
        checksum = (checksum << 5) - checksum + byte;
    }
    return checksum;
}

pair<uint16_t, uint16_t> LZ77::findLongestMatch(
    const vector<uint8_t>& data,
    size_t currentPos,
    uint32_t windowSize,
    uint32_t lookaheadSize) {

    uint16_t bestOffset = 0;
    uint16_t bestLength = 0;

    size_t searchStart = (currentPos > windowSize) ? currentPos - windowSize : 0;
    size_t maxLength = min(lookaheadSize, static_cast<uint32_t>(data.size() - currentPos));

    for (size_t offset = 1; offset <= currentPos - searchStart && offset <= 65535; offset++) {
        size_t matchPos = currentPos - offset;
        uint16_t length = 0;

        while (length < maxLength &&
            matchPos + length < data.size() &&
            data[matchPos + length] == data[currentPos + length]) {
            length++;
        }

        if (length > bestLength) {
            bestLength = length;
            bestOffset = static_cast<uint16_t>(offset);

            if (bestLength == maxLength) {
                break;
            }
        }
    }

    return { bestOffset, bestLength };
}

vector<LZ77Token> LZ77::compressTokens(const vector<uint8_t>& input,
    uint32_t windowSize,
    uint32_t lookaheadSize) {
    vector<LZ77Token> tokens;
    size_t pos = 0;
    size_t n = input.size();

    while (pos < n) {
        auto [offset, length] = findLongestMatch(input, pos, windowSize, lookaheadSize);

        if (length >= 3) {
            uint8_t nextChar = (pos + length < n) ? input[pos + length] : 0;
            tokens.push_back({ offset, static_cast<uint16_t>(length), nextChar });
            pos += length + 1;
        }
        else {
            tokens.push_back({ 0, 0, input[pos] });
            pos++;
        }
    }

    return tokens;
}

vector<uint8_t> LZ77::decompressTokens(const vector<LZ77Token>& tokens, size_t originalSize) {
    vector<uint8_t> output;
    output.reserve(originalSize);
```

```
    for (const auto& token : tokens) {
        if (token.offset == 0 && token.length == 0) {
            output.push_back(token.nextChar);
        }
        else {
            size_t startPos = output.size() - token.offset;
            for (uint16_t i = 0; i < token.length; i++) {
                output.push_back(output[startPos + i]);
            }
            if (token.nextChar != 0 || output.size() < originalSize) {
                output.push_back(token.nextChar);
            }
        }
    }

    if (output.size() > originalSize) {
        output.resize(originalSize);
    }

    return output;
}

vector<uint8_t> LZ77::tokensToBytes(const vector<LZ77Token>& tokens) {
    BitWriter writer;

    for (const auto& token : tokens) {
        if (token.offset == 0 && token.length == 0) {
            writer.writeBit(1);
            writer.writeBits(token.nextChar, 8);
        }
        else {
            writer.writeBit(0);
            writer.writeBits(token.offset, 16);
            writer.writeBits(token.length, 16);
            writer.writeBits(token.nextChar, 8);
        }
    }

    writer.flush();
    return writer.getData();
}

vector<LZ77Token> LZ77::bytesToTokens(const vector<uint8_t>& bytes) {
    BitReader reader(bytes);
    vector<LZ77Token> tokens;

    while (reader.hasMore()) {
        uint8_t flag = reader.readBit();

        if (flag == 1) {
            if (!reader.hasMore()) break;
            uint8_t nextChar = static_cast<uint8_t>(reader.readBits(8));
            tokens.push_back({ 0, 0, nextChar });
        }
        else {
            if (!reader.hasMore()) break;
            uint16_t offset = static_cast<uint16_t>(reader.readBits(16));
            if (!reader.hasMore()) break;
            uint16_t length = static_cast<uint16_t>(reader.readBits(16));
            if (!reader.hasMore()) break;
            uint8_t nextChar = static_cast<uint8_t>(reader.readBits(8));
            tokens.push_back({ offset, length, nextChar });
        }
    }

    return tokens;
}

vector<uint8_t> LZ77::encode(const vector<uint8_t>& input,
    uint32_t windowSize,
    uint32_t lookaheadSize) {
    if (input.empty()) {
        vector<uint8_t> result(sizeof(LZ77Header));
        LZ77Header header;
        header.magicNumber = LZ77_MAGIC;
        header.version = LZ77_VERSION;
        header.originalSize = 0;
        header.compressedSize = 0;
        header.windowSize = windowSize;
        header.lookaheadSize = lookaheadSize;
        header.checksum = 0;
        memcpy(result.data(), &header, sizeof(LZ77Header));
        return result;
    }

    vector<LZ77Token> tokens = compressTokens(input, windowSize, lookaheadSize);
    vector<uint8_t> compressedData = tokensToBytes(tokens);

    LZ77Header header;
    header.magicNumber = LZ77_MAGIC;
    header.version = LZ77_VERSION;
```

```
    header.originalSize = input.size();
    header.compressedSize = compressedData.size();
    header.windowSize = windowSize;
    header.lookaheadSize = lookaheadSize;
    header.checksum = calculateChecksum(compressedData);

    vector<uint8_t> result(sizeof(LZ77Header) + compressedData.size());
    memcpy(result.data(), &header, sizeof(LZ77Header));
    memcpy(result.data() + sizeof(LZ77Header), compressedData.data(), compressedData.size());

    return result;
}

vector<uint8_t> LZ77::decode(const vector<uint8_t>& input) {
    if (input.size() < sizeof(LZ77Header)) {
        return vector<uint8_t>();
    }

    LZ77Header header;
    memcpy(&header, input.data(), sizeof(LZ77Header));

    if (header.magicNumber != LZ77_MAGIC) {
        return vector<uint8_t>();
    }

    if (header.originalSize == 0) {
        return vector<uint8_t>();
    }

    vector<uint8_t> compressedData(input.begin() + sizeof(LZ77Header), input.end());

    uint32_t checksum = calculateChecksum(compressedData);
    if (checksum != header.checksum) {
        return vector<uint8_t>();
    }

    vector<LZ77Token> tokens = bytesToTokens(compressedData);

    return decompressTokens(tokens, header.originalSize);
}

bool LZ77::encodeFile(const string& inputPath,
    const string& outputPath,
    uint32_t windowSize,
    uint32_t lookaheadSize,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> encodedData = encode(inputData, windowSize, lookaheadSize);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool LZ77::decodeFile(const string& inputPath,
    const string& outputPath,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData);

    if (decodedData.empty()) {
        errorMsg = "Failed to decode data";
        return false;
    }

    ofstream outFile(outputPath, ios::binary);
```

```
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

LZ77Stats LZ77::compressFile(const string& inputPath,
    const string& outputPath,
    uint32_t windowSize,
    uint32_t lookaheadSize,
    string& errorMsg) {
    LZ77Stats stats;
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open file: " + inputPath;
        return stats;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();

    vector<uint8_t> encodedData = encode(inputData, windowSize, lookaheadSize);
    stats.compressedSize = encodedData.size();
    stats.compressionRatio = (double)stats.compressedSize / stats.originalSize * 100.0;
    stats.compressionFactor = (double)stats.originalSize / stats.compressedSize;
    stats.savingsPercent = 100.0 - stats.compressionRatio;

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return stats;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return stats;
}

bool LZ77::verifyCycle(const string& inputPath,
    const string& encodedPath,
    const string& decodedPath,
    uint32_t windowSize,
    uint32_t lookaheadSize,
    string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, windowSize, lookaheadSize, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)),
        istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)),
        istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            return false;
        }
    }

    return true;
}

void LZ77::printStats(const LZ77Stats& stats) {
    cout << "  Original: " << stats.originalSize << " bytes\n";
```

```
    cout << "  Compressed: " << stats.compressedSize << " bytes\n";
    cout << "  Ratio: " << fixed << setprecision(2)
        << stats.compressionRatio << "% of original\n";
    cout << "  Factor: " << fixed << setprecision(2)
        << stats.compressionFactor << ":1\n";
}
```

lz78.cpp

```
#include "lz78.h"
#include "bitstream.h"
#include <iostream>
#include <fstream>
#include <algorithm>
#include <cstring>
#include <map>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;


const uint32_t LZ78_MAGIC = 0x4C5A3738;
const uint8_t LZ78_VERSION = 1;

uint32_t LZ78::calculateChecksum(const vector<uint8_t>& data) {
    uint32_t checksum = 0;
    for (uint8_t byte : data) {
        checksum = (checksum << 5) - checksum + byte;
    }
    return checksum;
}

uint8_t LZ78::getIndexBitSize(uint32_t maxDictSize) {
    if (maxDictSize == 0) return 1;
    return static_cast<uint8_t>(ceil(log2(maxDictSize + 1)));
}

vector<uint8_t> LZ78::encodeTokens(const vector<uint8_t>& input, uint32_t maxDictSize) {
    BitWriter writer;


    map<vector<uint8_t>, uint32_t> dictionary;


    vector<uint8_t> empty;
    dictionary[empty] = 0;

    uint32_t nextIndex = 1;
    uint8_t indexBitSize = getIndexBitSize(maxDictSize);

    vector<uint8_t> current;

    size_t pos = 0;
    size_t n = input.size();

    while (pos < n) {
        current.push_back(input[pos]);


        if (dictionary.find(current) != dictionary.end() && pos + 1 < n) {
            pos++;
            continue;
        }


        vector<uint8_t> prefix(current.begin(), current.end() - 1);
        uint8_t lastChar = current.back();

        uint32_t prefixIndex = dictionary[prefix];


        writer.writeBits(prefixIndex, indexBitSize);

        writer.writeBits(lastChar, 8);


        if (nextIndex < maxDictSize) {
            dictionary[current] = nextIndex++;
        }


        current.clear();
        pos++;
    }

    writer.flush();
    return writer.getData();
}
```

```
vector<uint8_t> LZ78::decodeTokens(const vector<uint8_t>& compressed, size_t originalSize, uint32_t maxDictSize) {
    BitReader reader(compressed);
    vector<uint8_t> output;
    output.reserve(originalSize);


    map<uint32_t, vector<uint8_t>> dictionary;
    dictionary[0] = vector<uint8_t>();

    uint32_t nextIndex = 1;
    uint8_t indexBitSize = getIndexBitSize(maxDictSize);

    while (output.size() < originalSize && reader.hasMore()) {

        if (!reader.hasMore()) break;
        uint32_t prefixIndex = static_cast<uint32_t>(reader.readBits(indexBitSize));


        if (!reader.hasMore()) break;
        uint8_t nextChar = static_cast<uint8_t>(reader.readBits(8));


        vector<uint8_t> phrase;

        auto it = dictionary.find(prefixIndex);
        if (it != dictionary.end()) {
            phrase = it->second;
        }

        phrase.push_back(nextChar);


        for (uint8_t c : phrase) {
            if (output.size() < originalSize) {
                output.push_back(c);
            }
        }


        if (nextIndex < maxDictSize) {
            dictionary[nextIndex++] = phrase;
        }
    }

    return output;
}

vector<uint8_t> LZ78::encode(const vector<uint8_t>& input, uint32_t maxDictSize) {
    if (input.empty()) {
        vector<uint8_t> result(sizeof(LZ78Header));
        LZ78Header header;
        header.magicNumber = LZ78_MAGIC;
        header.version = LZ78_VERSION;
        header.originalSize = 0;
        header.compressedSize = 0;
        header.maxDictSize = maxDictSize;
        header.checksum = 0;
        memcpy(result.data(), &header, sizeof(LZ78Header));
        return result;
    }

    vector<uint8_t> compressedData = encodeTokens(input, maxDictSize);

    LZ78Header header;
    header.magicNumber = LZ78_MAGIC;
    header.version = LZ78_VERSION;
    header.originalSize = input.size();
    header.compressedSize = compressedData.size();
    header.maxDictSize = maxDictSize;
    header.checksum = calculateChecksum(compressedData);

    vector<uint8_t> result(sizeof(LZ78Header) + compressedData.size());
    memcpy(result.data(), &header, sizeof(LZ78Header));
    memcpy(result.data() + sizeof(LZ78Header), compressedData.data(), compressedData.size());

    return result;
}

vector<uint8_t> LZ78::decode(const vector<uint8_t>& input) {
    if (input.size() < sizeof(LZ78Header)) {
        return vector<uint8_t>();
    }

    LZ78Header header;
    memcpy(&header, input.data(), sizeof(LZ78Header));

    if (header.magicNumber != LZ78_MAGIC) {
        return vector<uint8_t>();
    }
```

```
    if (header.version != LZ78_VERSION) {
        return vector<uint8_t>();
    }

    if (header.originalSize == 0) {
        return vector<uint8_t>();
    }

    if (input.size() < sizeof(LZ78Header) + header.compressedSize) {
        return vector<uint8_t>();
    }

    vector<uint8_t> compressedData(input.begin() + sizeof(LZ78Header),
        input.begin() + sizeof(LZ78Header) + header.compressedSize);

    uint32_t checksum = calculateChecksum(compressedData);
    if (checksum != header.checksum) {
        return vector<uint8_t>();
    }

    return decodeTokens(compressedData, header.originalSize, header.maxDictSize);
}

bool LZ78::encodeFile(const string& inputPath,
    const string& outputPath,
    uint32_t maxDictSize,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> encodedData = encode(inputData, maxDictSize);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool LZ78::decodeFile(const string& inputPath,
    const string& outputPath,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData);

    if (decodedData.empty()) {
        errorMsg = "Failed to decode data";
        return false;
    }

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

LZ78Stats LZ78::compressFile(const string& inputPath,
    const string& outputPath,
    uint32_t maxDictSize,
    string& errorMsg) {
    LZ78Stats stats;
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
```

```
    if (!inFile.is_open()) {
        errorMsg = "Cannot open file: " + inputPath;
        return stats;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();

    map<vector<uint8_t>, uint32_t> dictionary;
    vector<uint8_t> empty;
    dictionary[empty] = 0;

    uint32_t nextIndex = 1;
    vector<uint8_t> current;

    size_t pos = 0;
    size_t n = inputData.size();

    stats.numLiterals = 0;
    stats.numPhrases = 0;
    double totalPhraseLength = 0;

    while (pos < n) {
        current.push_back(inputData[pos]);

        if (dictionary.find(current) != dictionary.end() && pos + 1 < n) {
            pos++;
            continue;
        }

        stats.numPhrases++;
        if (current.size() == 1) {
            stats.numLiterals++;
        }
        totalPhraseLength += current.size();

        if (nextIndex < maxDictSize) {
            dictionary[current] = nextIndex++;
        }

        current.clear();
        pos++;
    }

    if (stats.numPhrases > 0) {
        stats.averagePhraseLength = totalPhraseLength / stats.numPhrases;
    }
    stats.dictSize = min(nextIndex, maxDictSize);


    vector<uint8_t> encodedData = encode(inputData, maxDictSize);
    stats.compressedSize = encodedData.size();
    stats.compressionRatio = (double)stats.compressedSize / stats.originalSize * 100.0;
    stats.compressionFactor = (double)stats.originalSize / stats.compressedSize;
    stats.savingsPercent = 100.0 - stats.compressionRatio;


    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return stats;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return stats;
}

bool LZ78::verifyCycle(const string& inputPath,
    const string& encodedPath,
    const string& decodedPath,
    uint32_t maxDictSize,
    string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, maxDictSize, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
```

```
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)),
        istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)),
        istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        errorMsg = "Size mismatch";
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            errorMsg = "Data mismatch at position " + to_string(i);
            return false;
        }
    }

    return true;
}

void LZ78::printStats(const LZ78Stats& stats) {
    cout << "  Original: " << stats.originalSize << " bytes\n";
    cout << "  Compressed: " << stats.compressedSize << " bytes\n";
    cout << "  Ratio: " << fixed << setprecision(2)
        << stats.compressionRatio << "% of original\n";
    cout << "  Factor: " << fixed << setprecision(2)
        << stats.compressionFactor << ":1\n";
}
```

lzss.cpp

```
#include "lzss.h"
#include "bitstream.h"
#include <iostream>
#include <fstream>
#include <algorithm>
#include <cstring>
#include <vector>
#include <iomanip>

using namespace std;


const uint32_t LZSS_MAGIC = 0x4C5A5353;
const uint8_t LZSS_VERSION = 1;

uint32_t LZSS::calculateChecksum(const vector<uint8_t>& data) {
    uint32_t checksum = 0;
    for (uint8_t byte : data) {
        checksum = (checksum << 5) - checksum + byte;
    }
    return checksum;
}

pair<uint16_t, uint16_t> LZSS::findLongestMatch(
    const vector<uint8_t>& data,
    size_t currentPos,
    uint32_t windowSize,
    uint32_t lookaheadSize) {

    uint16_t bestOffset = 0;
    uint16_t bestLength = 0;

    size_t searchStart = (currentPos > windowSize) ? currentPos - windowSize : 0;
    size_t maxLength = min(lookaheadSize, static_cast<uint32_t>(data.size() - currentPos));

    for (size_t offset = 1; offset <= currentPos - searchStart && offset <= 65535; offset++) {
        size_t matchPos = currentPos - offset;
        uint16_t length = 0;

        while (length < maxLength &&
            matchPos + length < data.size() &&
            data[matchPos + length] == data[currentPos + length]) {
            length++;
        }

        if (length > bestLength) {
            bestLength = length;
            bestOffset = static_cast<uint16_t>(offset);

            if (bestLength == maxLength) {
                break;
            }
        }
    }

    return { bestOffset, bestLength };
}
```

```
vector<uint8_t> LZSS::encodeTokens(const vector<uint8_t>& input,
    uint32_t windowSize,
    uint32_t lookaheadSize) {
    BitWriter writer;

    size_t pos = 0;
    size_t n = input.size();

    while (pos < n) {
        auto [offset, length] = findLongestMatch(input, pos, windowSize, lookaheadSize);


        if (length >= 3) {

            writer.writeBit(0);
            writer.writeBits(offset, 16);
            writer.writeBits(length, 16);
            pos += length;
        }
        else {

            writer.writeBit(1);
            writer.writeBits(input[pos], 8);
            pos++;
        }
    }

    writer.flush();
    return writer.getData();
}

vector<uint8_t> LZSS::decodeTokens(const vector<uint8_t>& compressed, size_t originalSize) {
    BitReader reader(compressed);
    vector<uint8_t> output;
    output.reserve(originalSize);

    while (output.size() < originalSize && reader.hasMore()) {
        uint8_t flag = reader.readBit();

        if (flag == 1) {

            if (!reader.hasMore()) break;
            uint8_t literal = static_cast<uint8_t>(reader.readBits(8));
            output.push_back(literal);
        }
        else {

            if (!reader.hasMore()) break;
            uint16_t offset = static_cast<uint16_t>(reader.readBits(16));
            if (!reader.hasMore()) break;
            uint16_t length = static_cast<uint16_t>(reader.readBits(16));

            size_t startPos = output.size() - offset;
            for (uint16_t i = 0; i < length && output.size() < originalSize; i++) {
                output.push_back(output[startPos + i]);
            }
        }
    }

    return output;
}

vector<uint8_t> LZSS::encode(const vector<uint8_t>& input,
    uint32_t windowSize,
    uint32_t lookaheadSize) {
    if (input.empty()) {
        vector<uint8_t> result(sizeof(LZSSHeader));
        LZSSHeader header;
        header.magicNumber = LZSS_MAGIC;
        header.version = LZSS_VERSION;
        header.originalSize = 0;
        header.compressedSize = 0;
        header.windowSize = windowSize;
        header.lookaheadSize = lookaheadSize;
        header.checksum = 0;
        memcpy(result.data(), &header, sizeof(LZSSHeader));
        return result;
    }

    vector<uint8_t> compressedData = encodeTokens(input, windowSize, lookaheadSize);

    LZSSHeader header;
    header.magicNumber = LZSS_MAGIC;
    header.version = LZSS_VERSION;
    header.originalSize = input.size();
    header.compressedSize = compressedData.size();
    header.windowSize = windowSize;
    header.lookaheadSize = lookaheadSize;
    header.checksum = calculateChecksum(compressedData);
```

```
    vector<uint8_t> result(sizeof(LZSSHeader) + compressedData.size());
    memcpy(result.data(), &header, sizeof(LZSSHeader));
    memcpy(result.data() + sizeof(LZSSHeader), compressedData.data(), compressedData.size());

    return result;
}

vector<uint8_t> LZSS::decode(const vector<uint8_t>& input) {
    if (input.size() < sizeof(LZSSHeader)) {
        return vector<uint8_t>();
    }

    LZSSHeader header;
    memcpy(&header, input.data(), sizeof(LZSSHeader));

    if (header.magicNumber != LZSS_MAGIC) {
        return vector<uint8_t>();
    }

    if (header.originalSize == 0) {
        return vector<uint8_t>();
    }

    vector<uint8_t> compressedData(input.begin() + sizeof(LZSSHeader), input.end());

    uint32_t checksum = calculateChecksum(compressedData);
    if (checksum != header.checksum) {
        return vector<uint8_t>();
    }

    return decodeTokens(compressedData, header.originalSize);
}

bool LZSS::encodeFile(const string& inputPath,
    const string& outputPath,
    uint32_t windowSize,
    uint32_t lookaheadSize,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> encodedData = encode(inputData, windowSize, lookaheadSize);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool LZSS::decodeFile(const string& inputPath,
    const string& outputPath,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData);

    if (decodedData.empty()) {
        errorMsg = "Failed to decode data";
        return false;
    }

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();
```

```
    return true;
}

LZSSStats LZSS::compressFile(const string& inputPath,
    const string& outputPath,
    uint32_t windowSize,
    uint32_t lookaheadSize,
    string& errorMsg) {
    LZSSStats stats;
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open file: " + inputPath;
        return stats;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();


    vector<uint8_t> encodedData = encode(inputData, windowSize, lookaheadSize);
    stats.compressedSize = encodedData.size();
    stats.compressionRatio = (double)stats.compressedSize / stats.originalSize * 100.0;
    stats.compressionFactor = (double)stats.originalSize / stats.compressedSize;
    stats.savingsPercent = 100.0 - stats.compressionRatio;


    stats.numLiterals = 0;
    stats.numMatches = 0;

    size_t pos = 0;
    size_t n = inputData.size();
    double totalMatchLength = 0;
    double totalOffset = 0;

    while (pos < n) {
        auto [offset, length] = findLongestMatch(inputData, pos, windowSize, lookaheadSize);

        if (length >= 3) {
            stats.numMatches++;
            totalMatchLength += length;
            totalOffset += offset;
            pos += length;
        }
        else {
            stats.numLiterals++;
            pos++;
        }
    }

    if (stats.numMatches > 0) {
        stats.averageMatchLength = totalMatchLength / stats.numMatches;
        stats.averageOffset = totalOffset / stats.numMatches;
    }


    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return stats;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return stats;
}

bool LZSS::verifyCycle(const string& inputPath,
    const string& encodedPath,
    const string& decodedPath,
    uint32_t windowSize,
    uint32_t lookaheadSize,
    string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, windowSize, lookaheadSize, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
```

```
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)),
        istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)),
        istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            return false;
        }
    }

    return true;
}

void LZSS::printStats(const LZSSStats& stats) {
    cout << "  Original: " << stats.originalSize << " bytes\n";
    cout << "  Compressed: " << stats.compressedSize << " bytes\n";
    cout << "  Ratio: " << fixed << setprecision(2)
        << stats.compressionRatio << "% of original\n";
    cout << "  Factor: " << fixed << setprecision(2)
        << stats.compressionFactor << ":1\n";
}
```

lzw.cpp

```
#include "lzw.h"
#include "bitstream.h"
#include <iostream>
#include <fstream>
#include <algorithm>
#include <cstring>
#include <map>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;

const uint32_t LZW_MAGIC = 0x4C5A57;
const uint8_t LZW_VERSION = 1;

uint32_t LZW::calculateChecksum(const vector<uint8_t>& data) {
    uint32_t checksum = 0;
    for (uint8_t byte : data) {
        checksum = (checksum << 5) - checksum + byte;
    }
    return checksum;
}

uint8_t LZW::getCodeBitSize(uint32_t maxDictSize) {
    if (maxDictSize == 0) return 8;
    return static_cast<uint8_t>(ceil(log2(maxDictSize)));
}

map<vector<uint8_t>, uint32_t> LZW::buildInitialDictionary() {
    map<vector<uint8_t>, uint32_t> dictionary;


    for (int i = 0; i < 256; i++) {
        vector<uint8_t> entry;
        entry.push_back(static_cast<uint8_t>(i));
        dictionary[entry] = i;
    }

    return dictionary;
}

vector<uint8_t> LZW::encodeTokens(const vector<uint8_t>& input, uint32_t maxDictSize) {
    BitWriter writer;

    if (input.empty()) {
        return writer.getData();
    }


    map<vector<uint8_t>, uint32_t> dictionary = buildInitialDictionary();
    uint32_t nextCode = 256;
    uint8_t codeBitSize = getCodeBitSize(maxDictSize);

    vector<uint8_t> current;
    current.push_back(input[0]);

    size_t pos = 1;
```

```
    size_t n = input.size();

    while (pos < n) {
        vector<uint8_t> next = current;
        next.push_back(input[pos]);


        if (dictionary.find(next) != dictionary.end()) {
            current = next;
            pos++;
            continue;
        }

        uint32_t code = dictionary[current];
        writer.writeBits(code, codeBitSize);


        if (nextCode < maxDictSize) {
            dictionary[next] = nextCode++;
        }


        current.clear();
        current.push_back(input[pos]);
        pos++;
    }


    if (!current.empty()) {
        uint32_t code = dictionary[current];
        writer.writeBits(code, codeBitSize);
    }

    writer.flush();
    return writer.getData();
}

vector<uint8_t> LZW::decodeTokens(const vector<uint8_t>& compressed,
    size_t originalSize,
    uint32_t maxDictSize) {
    BitReader reader(compressed);
    vector<uint8_t> output;
    output.reserve(originalSize);

    if (compressed.empty()) {
        return output;
    }


    map<uint32_t, vector<uint8_t>> dictionary;


    for (int i = 0; i < 256; i++) {
        vector<uint8_t> entry;
        entry.push_back(static_cast<uint8_t>(i));
        dictionary[i] = entry;
    }

    uint32_t nextCode = 256;
    uint8_t codeBitSize = getCodeBitSize(maxDictSize);


    if (!reader.hasMore()) {
        return output;
    }

    uint32_t prevCode = static_cast<uint32_t>(reader.readBits(codeBitSize));
    vector<uint8_t> prevString = dictionary[prevCode];
    output.insert(output.end(), prevString.begin(), prevString.end());

    while (output.size() < originalSize && reader.hasMore()) {
        uint32_t currentCode = static_cast<uint32_t>(reader.readBits(codeBitSize));

        vector<uint8_t> currentString;


        if (dictionary.find(currentCode) != dictionary.end()) {
            currentString = dictionary[currentCode];
        }

        else if (currentCode == nextCode) {
            currentString = prevString;
            currentString.push_back(prevString[0]);
        }
        else {

            return vector<uint8_t>();
        }


        output.insert(output.end(), currentString.begin(), currentString.end());
```

```
        if (nextCode < maxDictSize) {
            vector<uint8_t> newString = prevString;
            newString.push_back(currentString[0]);
            dictionary[nextCode++] = newString;
        }

        prevString = currentString;
        prevCode = currentCode;
    }

    return output;
}

vector<uint8_t> LZW::encode(const vector<uint8_t>& input, uint32_t maxDictSize) {
    if (input.empty()) {
        vector<uint8_t> result(sizeof(LZWHeader));
        LZWHeader header;
        header.magicNumber = LZW_MAGIC;
        header.version = LZW_VERSION;
        header.originalSize = 0;
        header.compressedSize = 0;
        header.maxDictSize = maxDictSize;
        header.checksum = 0;
        memcpy(result.data(), &header, sizeof(LZWHeader));
        return result;
    }

    vector<uint8_t> compressedData = encodeTokens(input, maxDictSize);

    LZWHeader header;
    header.magicNumber = LZW_MAGIC;
    header.version = LZW_VERSION;
    header.originalSize = input.size();
    header.compressedSize = compressedData.size();
    header.maxDictSize = maxDictSize;
    header.checksum = calculateChecksum(compressedData);

    vector<uint8_t> result(sizeof(LZWHeader) + compressedData.size());
    memcpy(result.data(), &header, sizeof(LZWHeader));
    memcpy(result.data() + sizeof(LZWHeader), compressedData.data(), compressedData.size());

    return result;
}

vector<uint8_t> LZW::decode(const vector<uint8_t>& input) {
    if (input.size() < sizeof(LZWHeader)) {
        return vector<uint8_t>();
    }

    LZWHeader header;
    memcpy(&header, input.data(), sizeof(LZWHeader));

    if (header.magicNumber != LZW_MAGIC) {
        return vector<uint8_t>();
    }

    if (header.version != LZW_VERSION) {
        return vector<uint8_t>();
    }

    if (header.originalSize == 0) {
        return vector<uint8_t>();
    }

    if (input.size() < sizeof(LZWHeader) + header.compressedSize) {
        return vector<uint8_t>();
    }

    vector<uint8_t> compressedData(input.begin() + sizeof(LZWHeader),
        input.begin() + sizeof(LZWHeader) + header.compressedSize);

    uint32_t checksum = calculateChecksum(compressedData);
    if (checksum != header.checksum) {
        return vector<uint8_t>();
    }

    return decodeTokens(compressedData, header.originalSize, header.maxDictSize);
}

bool LZW::encodeFile(const string& inputPath,
    const string& outputPath,
    uint32_t maxDictSize,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }
```

```
    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> encodedData = encode(inputData, maxDictSize);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool LZW::decodeFile(const string& inputPath,
    const string& outputPath,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData);

    if (decodedData.empty()) {
        errorMsg = "Failed to decode data";
        return false;
    }

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

LZWStats LZW::compressFile(const string& inputPath,
    const string& outputPath,
    uint32_t maxDictSize,
    string& errorMsg) {
    LZWStats stats;
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open file: " + inputPath;
        return stats;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();


    map<vector<uint8_t>, uint32_t> dictionary = buildInitialDictionary();
    uint32_t nextCode = 256;

    vector<uint8_t> current;
    current.push_back(inputData[0]);

    size_t pos = 1;
    size_t n = inputData.size();
    stats.numCodes = 1;

    while (pos < n) {
        vector<uint8_t> next = current;
        next.push_back(inputData[pos]);

        if (dictionary.find(next) != dictionary.end()) {
            current = next;
            pos++;
            continue;
        }
```

```
        stats.numCodes++;
        if (nextCode < maxDictSize) {
            dictionary[next] = nextCode++;
        }

        current.clear();
        current.push_back(inputData[pos]);
        pos++;
    }

    stats.dictSize = min(nextCode, maxDictSize);
    uint8_t codeBitSize = getCodeBitSize(maxDictSize);
    stats.averageCodeLength = codeBitSize;


    vector<uint8_t> encodedData = encode(inputData, maxDictSize);
    stats.compressedSize = encodedData.size();
    stats.compressionRatio = (double)stats.compressedSize / stats.originalSize * 100.0;
    stats.compressionFactor = (double)stats.originalSize / stats.compressedSize;
    stats.savingsPercent = 100.0 - stats.compressionRatio;


    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return stats;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return stats;
}

bool LZW::verifyCycle(const string& inputPath,
    const string& encodedPath,
    const string& decodedPath,
    uint32_t maxDictSize,
    string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, maxDictSize, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)),
        istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)),
        istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        errorMsg = "Size mismatch";
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            errorMsg = "Data mismatch at position " + to_string(i);
            return false;
        }
    }

    return true;
}

void LZW::printStats(const LZWStats& stats) {
    cout << "  Original: " << stats.originalSize << " bytes\n";
    cout << "  Compressed: " << stats.compressedSize << " bytes\n";
    cout << "  Ratio: " << fixed << setprecision(2)
        << stats.compressionRatio << "% of original\n";
    cout << "  Factor: " << fixed << setprecision(2)
        << stats.compressionFactor << ":1\n";
}
```

main.cpp

```
#include "raw_convert.h"
#include "compressors.h"
#include "analysis.h"
#include <iostream>
```

```
#include <string>
#include <filesystem>
#include <limits>
#include <functional>
#include <vector>

using namespace std;

namespace fs = std::filesystem;

void printMainMenu() {
    cout << "\n========================================\n";
    cout << "Compression & Analysis Tool\n";
    cout << "========================================\n";
    cout << "1. Debug Mode (Algorithm Analysis)\n";
    cout << "2. Compressors Mode (Test All Compressors)\n";
    cout << "3. Exit\n";
    cout << "========================================\n";
    cout << "Choose mode (1-3): ";
}

void printCompressorsMenu() {
    cout << "\n========================================\n";
    cout << "Compressors Mode\n";
    cout << "========================================\n";
    cout << "1. Test all compressors on all files\n";
    cout << "2. Test specific compressor\n";
    cout << "3. Back to main menu\n";
    cout << "========================================\n";
    cout << "Choose option (1-3): ";
}

void printCompressorList() {
    cout << "\nAvailable compressors:\n";
    cout << "1. HA (Canonical Huffman)\n";
    cout << "2. RLE (Run-Length Encoding)\n";
    cout << "3. BWT + RLE\n";
    cout << "4. BWT + MTF + HA\n";
    cout << "5. BWT + MTF + RLE + HA\n";
    cout << "6. LZSS\n";
    cout << "7. LZSS + HA\n";
    cout << "8. LZW\n";
    cout << "9. LZW + HA\n";
    cout << "Enter choice (1-9): ";
}

void runCompressorsOnAllFiles(const string& inputDir, const string& outputDir) {
    vector<string> files = { "BW.raw", "Gray.raw", "RGB.raw", "enwik7.txt", "RusText.txt", "BIN.bin" };
    vector<CompressionResult> allResults;

    vector<pair<string, function<CompressionResult(const string&, const string&)>>> compressors;
    compressors.push_back({ "HA", Compressors::HA });
    compressors.push_back({ "RLE", Compressors::RLE });
    compressors.push_back({ "BWT+RLE", Compressors::BWTRLE });
    compressors.push_back({ "BWT+MTF+HA", Compressors::BWTMTFHA });
    compressors.push_back({ "BWT+MTF+RLE+HA", Compressors::BWTMTFRLEHA });
    compressors.push_back({ "LZSS", Compressors::LZSS });
    compressors.push_back({ "LZSS+HA", Compressors::LZSSHA });
    compressors.push_back({ "LZW", Compressors::LZW });
    compressors.push_back({ "LZW+HA", Compressors::LZWHA });

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;
        if (!fs::exists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n";
            continue;
        }

        cout << "\n========================================\n";
        cout << "Testing on: " << filename << "\n";
        cout << "========================================\n";

        string basePath = outputDir + "\\" + filename;

        for (size_t i = 0; i < compressors.size(); i++) {
            cout << "\n" << compressors[i].first << ":\n";
            cout << "----------------------------------------\n";
            CompressionResult result = compressors[i].second(inputPath, basePath + "_" + compressors[i].first);
            allResults.push_back(result);
            Compressors::printResult(result);
        }
    }

    Compressors::printSummary(allResults);
}

void runSpecificCompressor(const string& inputDir, const string& outputDir) {
    printCompressorList();
    int compChoice;
    cin >> compChoice;
```

```
    if (compChoice < 1 || compChoice > 9) {
        cout << "Invalid choice\n";
        return;
    }

    Analysis::printFileSelectionMenu();
    int fileChoice;
    cin >> fileChoice;

    if (fileChoice == 8) {
        cout << "Operation cancelled\n";
        return;
    }

    vector<string> allFiles = { "BW.raw", "Gray.raw", "RGB.raw", "enwik7.txt", "RusText.txt", "BIN.bin" };
    vector<string> files;

    if (fileChoice == 7) {
        files = allFiles;
    }
    else if (fileChoice >= 1 && fileChoice <= 6) {
        files.push_back(allFiles[fileChoice - 1]);
    }
    else {
        cout << "Invalid choice\n";
        return;
    }

    vector<pair<string, function<CompressionResult(const string&, const string&)>>> compressors;
    compressors.push_back({ "HA", Compressors::HA });
    compressors.push_back({ "RLE", Compressors::RLE });
    compressors.push_back({ "BWT+RLE", Compressors::BWTRLE });
    compressors.push_back({ "BWT+MTF+HA", Compressors::BWTMTFHA });
    compressors.push_back({ "BWT+MTF+RLE+HA", Compressors::BWTMTFRLEHA });
    compressors.push_back({ "LZSS", Compressors::LZSS });
    compressors.push_back({ "LZSS+HA", Compressors::LZSSHA });
    compressors.push_back({ "LZW", Compressors::LZW });
    compressors.push_back({ "LZW+HA", Compressors::LZWHA });

    auto comp = compressors[compChoice - 1];
    vector<CompressionResult> results;

    for (const auto& filename : files) {
        string inputPath = inputDir + "\\" + filename;
        if (!fs::exists(inputPath)) {
            cout << "File not found: " << filename << " - skipping\n";
            continue;
        }

        cout << "\n========================================\n";
        cout << comp.first << " on " << filename << "\n";
        cout << "========================================\n";

        string basePath = outputDir + "\\" + filename + "_" + comp.first;
        CompressionResult result = comp.second(inputPath, basePath);
        results.push_back(result);
        Compressors::printResult(result);
    }

    Compressors::printSummary(results);
}

void printDebugMenu() {
    cout << "\n========================================\n";
    cout << "Debug Mode - Algorithm Analysis\n";
    cout << "========================================\n";
    cout << "1. RLE Compression Analysis\n";
    cout << "2. Entropy Calculation\n";
    cout << "3. MTF Transform Analysis\n";
    cout << "4. Huffman Compression Analysis\n";
    cout << "5. Canonical Huffman Compression\n";
    cout << "6. BWT Transform Test (banana)\n";
    cout << "7. BWT + RLE Combined Compression\n";
    cout << "8. Suffix Array to BWT Last Column Demo\n";
    cout << "9. Efficient BWT (Suffix Array Based)\n";
    cout << "10. LZ77 Compression Analysis\n";
    cout << "11. LZSS Compression Analysis\n";
    cout << "12. LZ78 Compression Analysis\n";
    cout << "13. LZW Compression Analysis\n";
    cout << "14. Run ALL Analyses on ALL Files\n";
    cout << "15. Back to main menu\n";
    cout << "========================================\n";
    cout << "Choose algorithm (1-15): ";
}

int main() {
    const string BASE_PATH = "C:\\Users\\chivo\\Documents\\AISD_2.2_n1";
    const string INPUT_DIR = BASE_PATH + "\\Input files";
    const string ENCODED_DIR = BASE_PATH + "\\Encoded files";
    const string DECODED_DIR = BASE_PATH + "\\Decoded files";
    const string COMPRESSOR_DIR = BASE_PATH + "\\Compressor results";
```

```
fs::create_directories(INPUT_DIR);
fs::create_directories(ENCODED_DIR);
fs::create_directories(DECODED_DIR);
fs::create_directories(COMPRESSOR_DIR);

cout << "Converting images to RAW format...\n";
cout << "----------------------------------------\n";
if (!convertImagesToRaw(INPUT_DIR, INPUT_DIR)) {
    cout << "Error: Failed to convert images\n";
    return 1;
}
cout << "Images converted successfully!\n";
cout << "\n========================================\n";
cout << "File Storage Locations:\n";
cout << "========================================\n";
cout << "Input files:     " << INPUT_DIR << "\n";
cout << "Encoded files:   " << ENCODED_DIR << "\n";
cout << "Decoded files:   " << DECODED_DIR << "\n";
cout << "Compressor results: " << COMPRESSOR_DIR << "\n";
cout << "========================================\n";
cout << "Note: All encoded and decoded files are saved permanently\n";
cout << "      in the respective directories for later analysis.\n\n";

while (true) {
    printMainMenu();
    int mode;
    cin >> mode;

    if (cin.fail()) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Invalid input. Please enter a number.\n";
        continue;
    }

    if (mode == 3) {
        cout << "Exiting program...\n";
        break;
    }

    if (mode == 2) {
        while (true) {
            printCompressorsMenu();
            int compMode;
            cin >> compMode;

            if (compMode == 3) break;

            if (compMode == 1) {
                runCompressorsOnAllFiles(INPUT_DIR, COMPRESSOR_DIR);
            }
            else if (compMode == 2) {
                runSpecificCompressor(INPUT_DIR, COMPRESSOR_DIR);
            }
            else {
                cout << "Invalid choice\n";
            }
        }
    }
    else if (mode == 1) {
        while (true) {
            printDebugMenu();
            int debugChoice;
            cin >> debugChoice;

            if (debugChoice == 15) break;

            if (debugChoice == 14) {
                Analysis::runAllAnalyses(INPUT_DIR, ENCODED_DIR, DECODED_DIR);
            }
            else {
                switch (debugChoice) {
                case 1: Analysis::rleAnalysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                case 2: Analysis::entropyAnalysis(INPUT_DIR); break;
                case 3: Analysis::mtfAnalysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                case 4: Analysis::huffmanAnalysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                case 5: Analysis::huffmanCanonicalAnalysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                case 6: Analysis::testOnBanana(); break;
                case 7: Analysis::testBWTAndRLE(INPUT_DIR); break;
                case 8: Analysis::suffixArrayToBWTDemo(INPUT_DIR); break;
                case 9: Analysis::testEfficientBWT(INPUT_DIR); break;
                case 10: Analysis::lz77Analysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                case 11: Analysis::lzssAnalysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                case 12: Analysis::lz78Analysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                case 13: Analysis::lzwAnalysis(INPUT_DIR, ENCODED_DIR, DECODED_DIR); break;
                default: cout << "Invalid choice\n"; break;
                }
            }
        }
    }
```

```
        else {
            cout << "Invalid choice. Please select 1-3.\n";
        }
    }

    return 0;
}
```

mtf.cpp

```
#include "mtf.h"
#include <vector>
#include <fstream>
#include <algorithm>
#include <cstdint>

using namespace std;

vector<uint8_t> MTF::encode(const vector<uint8_t>& input) {
    vector<uint8_t> output;

    if (input.empty()) {
        return output;
    }

    vector<uint8_t> symbolList(256);
    for (int i = 0; i < 256; i++) {
        symbolList[i] = i;
    }

    for (size_t i = 0; i < input.size(); i++) {
        uint8_t symbol = input[i];

        size_t pos = 0;
        while (pos < symbolList.size() && symbolList[pos] != symbol) {
            pos++;
        }

        output.push_back(static_cast<uint8_t>(pos));

        for (size_t j = pos; j > 0; j--) {
            symbolList[j] = symbolList[j - 1];
        }
        symbolList[0] = symbol;
    }

    return output;
}

vector<uint8_t> MTF::decode(const vector<uint8_t>& input) {
    vector<uint8_t> output;

    if (input.empty()) {
        return output;
    }

    vector<uint8_t> symbolList(256);
    for (int i = 0; i < 256; i++) {
        symbolList[i] = i;
    }

    for (size_t i = 0; i < input.size(); i++) {
        uint8_t index = input[i];

        if (index >= symbolList.size()) {
            return output;
        }

        uint8_t symbol = symbolList[index];
        output.push_back(symbol);

        for (size_t j = index; j > 0; j--) {
            symbolList[j] = symbolList[j - 1];
        }
        symbolList[0] = symbol;
    }

    return output;
}

bool MTF::encodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();
```

```
    vector<uint8_t> encodedData = encode(inputData);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool MTF::decodeFile(const string& inputPath, const string& outputPath, string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    vector<uint8_t> decodedData = decode(inputData);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

bool MTF::verifyCycle(const string& inputPath, const string& encodedPath,
    const string& decodedPath, string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)),
        istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)),
        istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            return false;
        }
    }

    return true;
}
```

raw_convert.cpp

```
#include "raw_convert.h"
#include <iostream>
#include <fstream>
#include <cstring>
#include <sys/stat.h>

#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"

using namespace std;
```

```
bool jpgToRaw(const string& inputFile, RawResult& out) {
    struct stat st;
    if (stat(inputFile.c_str(), &st) != 0) {
        return false;
    }
    out.jpgSize = st.st_size;

    int w, h, c;
    unsigned char* img = stbi_load(inputFile.c_str(), &w, &h, &c, 0);
    if (!img) {
        return false;
    }

    out.width = w;
    out.height = h;
    out.pixels = w * h;

    if (c == 1) {
        bool isBlackWhite = true;
        for (uint32_t i = 0; i < out.pixels; i++) {
            uint8_t val = img[i];
            if (val > 10 && val < 245) {
                isBlackWhite = false;
                break;
            }
        }

        if (isBlackWhite) {
            out.type = 1;
            out.data.resize(out.pixels);
            for (uint32_t i = 0; i < out.pixels; i++) {
                out.data[i] = (img[i] > 127) ? 255 : 0;
            }
        }
        else {
            out.type = 2;
            out.data.assign(img, img + out.pixels);
        }
    }
    else {
        vector<uint8_t> gray(out.pixels);
        for (uint32_t i = 0; i < out.pixels; i++) {
            gray[i] = (img[i * 3] + img[i * 3 + 1] + img[i * 3 + 2]) / 3;
        }

        bool isBlackWhite = true;
        int blackCount = 0, whiteCount = 0;

        for (uint32_t i = 0; i < out.pixels; i++) {
            if (gray[i] < 30) blackCount++;
            else if (gray[i] > 225) whiteCount++;
            else {
                isBlackWhite = false;
                break;
            }
        }

        if (isBlackWhite && (blackCount + whiteCount) > out.pixels * 0.95) {
            out.type = 1;
            out.data.resize(out.pixels);
            for (uint32_t i = 0; i < out.pixels; i++) {
                out.data[i] = (gray[i] > 127) ? 255 : 0;
            }
        }
        else {
            bool isGrayscale = true;
            for (uint32_t i = 0; i < out.pixels && isGrayscale; i++) {
                uint8_t r = img[i * 3], g = img[i * 3 + 1], b = img[i * 3 + 2];
                if (r != g || g != b) {
                    isGrayscale = false;
                }
            }

            if (isGrayscale) {
                out.type = 2;
                out.data = gray;
            }
            else {
                out.type = 3;
                out.data.assign(img, img + out.pixels * 3);
            }
        }
    }

    out.rawSize = 5 + out.data.size();

    stbi_image_free(img);
    return true;
}

bool convertImagesToRaw(const string& inputDir, const string& outputDir) {
```

```
    const char* imageNames[] = { "BW.jpg", "Gray.jpg", "RGB.jpg" };
    const char* imageTypes[] = { "Black & White", "Grayscale", "Color" };

    for (int i = 0; i < 3; i++) {
        string inputFile = inputDir + "\\" + imageNames[i];
        string outputFile = outputDir + "\\" + string(imageNames[i]).substr(0, string(imageNames[i]).find_last_of('.')) +
".raw";

        RawResult result;
        if (jpgToRaw(inputFile, result)) {
            ofstream f(outputFile, ios::binary);
            if (f.is_open()) {
                f.put(result.type);
                f.write((char*)&result.pixels, 4);
                f.write((char*)result.data.data(), result.data.size());
                f.close();

                cout << imageNames[i] << " -> " << imageTypes[i] << "\n";
                cout << "  Resolution: " << result.width << "x" << result.height << "\n";
                cout << "  JPG size: " << result.jpgSize << " bytes\n";
                cout << "  RAW size: " << result.rawSize << " bytes (header + pixel data)\n";
                cout << "  Pixel data: " << result.data.size() << " bytes\n";
                cout << "  JPG is " << (float)result.jpgSize / result.rawSize * 100 << "% of RAW\n";
                cout << "  JPG compression ratio: " << (float)result.rawSize / result.jpgSize << ":1\n\n";
            }
        }
        else {
            cout << "Failed to convert " << imageNames[i] << "\n";
            return false;
        }
    }

    return true;
}

bool extractPixelData(const string& rawPath, vector<uint8_t>& pixelData, uint8_t& type, uint32_t& pixels) {
    ifstream f(rawPath, ios::binary);
    if (!f.is_open()) {
        return false;
    }

    f.read((char*)&type, 1);
    f.read((char*)&pixels, 4);

    pixelData.clear();
    pixelData.assign((istreambuf_iterator<char>(f)), istreambuf_iterator<char>());
    f.close();

    return true;
}
```

rle.cpp

```
#include "rle.h"
#include "bitstream.h"
#include <iostream>
#include <fstream>
#include <vector>
#include <cstdint>
#include <cstring>
#include <algorithm>
#include <iomanip>

using namespace std;

vector<uint8_t> RLE::encode(const vector<uint8_t>& input, uint8_t Ms, uint8_t Mc) {
    BitWriter writer;

    if (input.empty() || Ms == 0 || Mc == 0) {
        return writer.getData();
    }

    size_t i = 0;
    size_t dataSize = input.size();

    while (i < dataSize) {
        size_t remaining = dataSize - i;

        if (remaining >= Ms) {
            size_t runLen = 1;
            uint64_t maxRun = (1ULL << (Mc * 8)) - 1;

            while (i + (runLen + 1) * Ms <= dataSize && runLen < maxRun) {
                bool match = true;
                for (uint8_t k = 0; k < Ms && match; k++) {
                    if (input[i + k] != input[i + runLen * Ms + k]) {
                        match = false;
                    }
                }
```

```
                if (!match) break;
                runLen++;
            }

            if (runLen >= 2) {
                writer.writeBit(0);
                writer.writeBits(runLen, Mc * 8);
                for (uint8_t k = 0; k < Ms; k++) {
                    writer.writeBits(input[i + k], 8);
                }
                i += runLen * Ms;
                continue;
            }
        }

        uint64_t maxLiteral = (1ULL << (Mc * 8)) - 1;
        uint64_t literalLen = 1;

        while (i + (literalLen + 1) * Ms <= dataSize && literalLen < maxLiteral) {
            if (remaining >= (literalLen + 2) * Ms) {
                bool nextIsRun = true;
                for (uint8_t k = 0; k < Ms; k++) {
                    if (input[i + literalLen * Ms + k] !=
                        input[i + (literalLen + 1) * Ms + k]) {
                        nextIsRun = false;
                        break;
                    }
                }
                if (nextIsRun) break;
            }
            literalLen++;
        }

        writer.writeBit(1);
        writer.writeBits(literalLen, Mc * 8);
        for (uint64_t j = 0; j < literalLen; j++) {
            for (uint8_t k = 0; k < Ms; k++) {
                writer.writeBits(input[i + j * Ms + k], 8);
            }
        }
        i += literalLen * Ms;
    }

    writer.flush();
    return writer.getData();
}

vector<uint8_t> RLE::decode(const vector<uint8_t>& input, uint8_t Ms, uint8_t Mc) {
    BitReader reader(input);
    vector<uint8_t> output;

    if (input.empty() || Ms == 0 || Mc == 0) {
        return output;
    }

    while (reader.hasMore()) {
        uint8_t controlBit = reader.readBit();
        uint64_t length = reader.readBits(Mc * 8);

        if (controlBit == 1) {
            for (uint64_t j = 0; j < length; j++) {
                for (uint8_t k = 0; k < Ms; k++) {
                    if (!reader.hasMore()) break;
                    output.push_back(static_cast<uint8_t>(reader.readBits(8)));
                }
            }
        }
        else {
            vector<uint8_t> symbol;
            for (uint8_t k = 0; k < Ms; k++) {
                if (!reader.hasMore()) break;
                symbol.push_back(static_cast<uint8_t>(reader.readBits(8)));
            }
            for (uint64_t j = 0; j < length; j++) {
                output.insert(output.end(), symbol.begin(), symbol.end());
            }
        }
    }

    return output;
}

bool RLE::encodeFile(const string& inputPath,
    const string& outputPath,
    uint8_t Ms,
    uint8_t Mc,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open input file: " + inputPath;
        return false;
```

```
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    size_t originalSize = inputData.size();

    vector<uint8_t> encodedData = encode(inputData, Ms, Mc);

    RLEHeader header;
    header.Ms = Ms;
    header.Mc = Mc;
    header.originalSize = originalSize;
    header.encodedSize = encodedData.size();

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(&header.Ms), sizeof(header.Ms));
    outFile.write(reinterpret_cast<const char*>(&header.Mc), sizeof(header.Mc));
    outFile.write(reinterpret_cast<const char*>(&header.originalSize), sizeof(header.originalSize));
    outFile.write(reinterpret_cast<const char*>(&header.encodedSize), sizeof(header.encodedSize));
    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return true;
}

bool RLE::decodeFile(const string& inputPath,
    const string& outputPath,
    string& errorMsg) {
    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open encoded file: " + inputPath;
        return false;
    }

    RLEHeader header;
    inFile.read(reinterpret_cast<char*>(&header.Ms), sizeof(header.Ms));
    inFile.read(reinterpret_cast<char*>(&header.Mc), sizeof(header.Mc));
    inFile.read(reinterpret_cast<char*>(&header.originalSize), sizeof(header.originalSize));
    inFile.read(reinterpret_cast<char*>(&header.encodedSize), sizeof(header.encodedSize));

    if (inFile.fail()) {
        errorMsg = "Failed to read header";
        return false;
    }

    vector<uint8_t> encodedData(header.encodedSize);
    inFile.read(reinterpret_cast<char*>(encodedData.data()), header.encodedSize);
    inFile.close();

    vector<uint8_t> decodedData = decode(encodedData, header.Ms, header.Mc);

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create decoded file: " + outputPath;
        return false;
    }

    outFile.write(reinterpret_cast<const char*>(decodedData.data()), decodedData.size());
    outFile.close();

    return true;
}

CompressionStats RLE::analyzeFile(const string& inputPath, uint8_t Ms, uint8_t Mc, string& errorMsg) {
    CompressionStats stats;
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open file: " + inputPath;
        return stats;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();

    size_t totalRuns = 0;
    size_t totalRunLength = 0;
    size_t currentRun = 1;

    for (size_t i = Ms; i < inputData.size(); i += Ms) {
```

```
        bool same = true;
        for (uint8_t k = 0; k < Ms && same; k++) {
            if (inputData[i - Ms + k] != inputData[i + k]) {
                same = false;
            }
        }

        if (same) {
            currentRun++;
        }
        else {
            if (currentRun > 1) {
                totalRuns++;
                totalRunLength += currentRun;
            }
            currentRun = 1;
        }
    }

    if (currentRun > 1) {
        totalRuns++;
        totalRunLength += currentRun;
    }

    size_t totalSymbols = inputData.size() / Ms;

    if (totalRuns > 0) {
        double encodedEstimate = totalRuns * (Mc + Ms);
        double literalEstimate = (totalSymbols - totalRunLength) * (1 + Ms);
        double totalEstimate = encodedEstimate + literalEstimate;
        stats.compressionRatio = totalEstimate / inputData.size() * 100.0;
    }
    else {
        stats.compressionRatio = 100.0;
    }

    stats.compressionFactor = 100.0 / stats.compressionRatio;
    stats.savingsPercent = 100.0 - stats.compressionRatio;

    return stats;
}

CompressionStats RLE::compressFile(const string& inputPath,
    const string& outputPath,
    uint8_t Ms,
    uint8_t Mc,
    string& errorMsg) {
    CompressionStats stats;
    stats.filename = inputPath.substr(inputPath.find_last_of("/\\") + 1);

    ifstream inFile(inputPath, ios::binary);
    if (!inFile.is_open()) {
        errorMsg = "Cannot open file: " + inputPath;
        return stats;
    }

    vector<uint8_t> inputData((istreambuf_iterator<char>(inFile)),
        istreambuf_iterator<char>());
    inFile.close();

    stats.originalSize = inputData.size();

    vector<uint8_t> encodedData = encode(inputData, Ms, Mc);

    RLEHeader header;
    header.Ms = Ms;
    header.Mc = Mc;
    header.originalSize = stats.originalSize;
    header.encodedSize = encodedData.size();

    stats.encodedSize = sizeof(header) + encodedData.size();
    stats.compressionRatio = (double)stats.encodedSize / stats.originalSize * 100.0;
    stats.compressionFactor = (double)stats.originalSize / stats.encodedSize;
    stats.savingsPercent = 100.0 - stats.compressionRatio;

    ofstream outFile(outputPath, ios::binary);
    if (!outFile.is_open()) {
        errorMsg = "Cannot create output file: " + outputPath;
        return stats;
    }

    outFile.write(reinterpret_cast<const char*>(&header.Ms), sizeof(header.Ms));
    outFile.write(reinterpret_cast<const char*>(&header.Mc), sizeof(header.Mc));
    outFile.write(reinterpret_cast<const char*>(&header.originalSize), sizeof(header.originalSize));
    outFile.write(reinterpret_cast<const char*>(&header.encodedSize), sizeof(header.encodedSize));
    outFile.write(reinterpret_cast<const char*>(encodedData.data()), encodedData.size());
    outFile.close();

    return stats;
}
```

```
double RLE::getCompressionRatio(size_t originalSize, size_t encodedSize) {
    if (originalSize == 0) return 0.0;
    return (double)encodedSize / originalSize * 100.0;
}

bool RLE::verifyCycle(const string& inputPath,
    const string& encodedPath,
    const string& decodedPath,
    uint8_t Ms,
    uint8_t Mc,
    string& errorMsg) {
    if (!encodeFile(inputPath, encodedPath, Ms, Mc, errorMsg)) {
        return false;
    }

    if (!decodeFile(encodedPath, decodedPath, errorMsg)) {
        return false;
    }

    ifstream origFile(inputPath, ios::binary);
    ifstream decFile(decodedPath, ios::binary);

    if (!origFile.is_open() || !decFile.is_open()) {
        errorMsg = "Cannot open files for comparison";
        return false;
    }

    vector<uint8_t> origData((istreambuf_iterator<char>(origFile)),
        istreambuf_iterator<char>());
    vector<uint8_t> decData((istreambuf_iterator<char>(decFile)),
        istreambuf_iterator<char>());

    if (origData.size() != decData.size()) {
        return false;
    }

    for (size_t i = 0; i < origData.size(); i++) {
        if (origData[i] != decData[i]) {
            return false;
        }
    }

    return true;
}

void RLE::printStats(const CompressionStats& stats) {
    cout << "  Original: " << stats.originalSize << " bytes\n";
    cout << "  Encoded:  " << stats.encodedSize << " bytes\n";
    cout << "  Ratio:    " << fixed << setprecision(2) << stats.compressionRatio << "% of original\n";
    cout << "  Factor:   " << fixed << setprecision(2) << stats.compressionFactor << ":1\n";
}
```